PROJECT SENTINEL

Microsoft 365 Governance Intelligence Extraction Tool

SYSTEM MAINTENANCE DOCUMENT

Industry Partner

Christopher McNaughton - SECMON1

Team Shadow Stacks - La Trobe University

Prepared by:

Lahari Borra - Documentation Specialist (22134698)

Raj Kumar Mustyala - Team Lead (22234543)

Yaswanth Kanderi - Backend Developer (22519671)

Vanka Manjunath - API Developer (22197477)

Bharanee Raghav Murru - Dashboard Developer (22003177)

Jaswanthi Palleti - Security Analyst (22115229)

Copyright and Warranty Information

This document and all the information in it belong to Team Shadow Stacks at La Trobe University. It was created as part of the CSE5IDP Capstone Industry Development Program. You cannot copy, share, or republish any part of this document without written permission from the authors.

All product names, company names, and trademarks mentioned in this document belong to their respective owners.

This document is provided as-is. The team makes no guarantees about the accuracy or completeness of the information. The authors are not responsible for any errors, omissions, or damages that result from using this documentation or the system it describes.

This document and Project Sentinel were built purely for educational purposes as part of the university program. The tool is not intended for production use without further review, security hardening, and testing in a real enterprise environment.

The developer tenant used throughout this project (Sentinals.onmicrosoft.com) contains only synthetic test data. No real personal, financial, or confidential data was used at any point during development or testing.

Table Of Content:

1. System Overview

1.1 Purpose

Project Sentinel is a tool our team built for SECMON1 as part of La Trobe University's Capstone Industry Development Program. The goal was to give the industry partner a way to automatically pull security and governance data out of a Microsoft 365 tenant and turn it into something useful for monitoring and investigation.

Microsoft 365 generates a lot of valuable security signals on its own things like who signed in and from where, what files exist in SharePoint, who has permission to access those files, and whether those files have been properly classified. All of that information exists, but it is spread across multiple Microsoft APIs and does not come together in one place automatically. Project Sentinel bridges that gap. It connects to Microsoft's APIs using a service account, pulls the data, runs a governance analysis, and writes everything out into consistent JSON files that the Shadow Sight platform can ingest and use for monitoring.

This document is the main technical reference for anyone who needs to maintain, extend, or hand over the system. It covers what the system does, how it is designed, what it connects to, how to run it, how we tested it, and what the team built across six sprints.

1.2 Intended Audience

This document is written for four groups. Security analysts at SECMON1 who will run the tool and use the data it produces. Backend developers who need to maintain or extend the code.

1.3 Scope

Project Sentinel connects to the Microsoft 365 developer tenant

(Sentinals.onmicrosft.com) using a noninteractive service account authenticated via OAuth 2.0. Every day at 8am, a Power Automate scheduled flow (Project-Sentinel-Daily-Extraction) triggers the extraction engine automatically. Five extraction modules run and pull data from four Microsoft API surfaces sign-in audit logs from the Unified Audit Log, file metadata from SharePoint document libraries, sharing permissions across all drives, and sensitivity label data from Microsoft Purview and DLP Policy. DLP policy (Sentinel-Teams-DLP-Policy) is configured and actively monitoring the tenant in simulation mode, detecting sensitive information types such as Australia Tax File Numbers.

Once the data is collected, each module runs a governance analysis on its own dataset checking for things like missing sensitivity labels, anonymous sharing links, inactive files, and broken permission inheritance. The results are written to nine JSON output files using a consistent versioned schema (version 2.0.0, 95 fields across 9 categories). Every output file includes a provenance envelope at the top so downstream systems like ShadowSight

can validate the file before importing it. A cumulative run log is also updated in extraction_history.xlsx after every run.

Power BI dashboard connects directly to the JSON output files and gives the security analyst a visual view of the governance data across five pages Audit Monitoring, File Insights, Governance and Compliance, Tenant Overview, and File Activity.

1.4 Document Conventions

Code, file names, and command-line text are shown in Courier New font for example, main.py or /output/. API endpoints are shown with their HTTP method and path for example, GET /auditLogs/signIns. The Microsoft identity service is called Azure Entra ID throughout this document. The developer tenant is Sentinals.onmicrosoft.com and the scheduled flow is named Project-Sentinel-DailyExtraction both names are used exactly as they appear in the portals. Code, file names, and command-line text are shown in Courier New font for example, main.py or /output/. API endpoints are shown with their HTTP method and path for example, GET /auditLogs/signIns. The Microsoft identity service is called Azure Entra ID throughout this document. The developer tenant is Sentinals.onmicrosoft.com and the scheduled flow is named Project-Sentinel-DailyExtraction - both names are used exactly as they appear in the portals.

1.5 Version History

Table 1.5 Document Version History

Version	Date	Author	Changes
1.0	Sprint 3 (April 2026)	Lahari Borra	Initial document structure, output schema, API permissions mapping
2.0	Sprint 4 (April–May 2026)	Lahari Borra	System design, deployment guide, usability trial results added
3.0	Sprint 5 (May 2026)	Lahari Borra	Live extraction data, dashboard screenshots, project history
4.0 (Final)	Sprint 6 (May 2026)	Lahari Borra	All sections reviewed and updated, architecture diagrams embedded, Sprint 6 dashboard and bug fixes documented, vendor documentation and methodology added.

2. Software Design Scope

2.1 Major Features

Project Sentinel does five things in every extraction run. First, it authenticates to the Microsoft 365 tenant using a service account nobody needs to be logged in when it runs. Second, it pulls data from four API sources: sign-in audit logs, SharePoint file metadata, sharing permissions, and Purview sensitivity labels. Third, it runs a governance analysis on each dataset to flag problems like missing sensitivity labels, broad permission grants, or sign-in failures above a threshold. Fourth, it exports everything to JSON files in a consistent, versioned format with a provenance envelope at the top of each file. Fifth, it logs every run to extraction_history.xlsx so there is a clear record of what was extracted and when.

2.2 Technology Stack and Design Justification

Table 2.2 Technology Stack

Component	Technology	Justification
Language	Python 3.11	Strong library support for HTTP, JSON, and Excel. Team had the most Python experience.
Auth library	MSAL 1.24+	Microsoft's own library for OAuth 2.0. Handles token requests and caching reliably.
Primary API	Microsoft Graph API v1.0	Microsoft's unified REST API for all MS365 services. Stable and well-documented.
Extended API	Microsoft Graph API beta	Required for sensitivity labels and retention labels- not yet in v1.0.
SharePoint access	SharePoint REST API v1.0	Needed for site and document library enumeration alongside Graph.
Compliance data	Purview Compliance API	Only way to access DLP matches and sensitive information types.
Cloud hosting	Microsoft Power Automate	Scheduled flow (Project-Sentinel-Daily-Extraction) triggers the pipeline daily at 08:00 AM. No additional Azure infrastructure required.
Scheduling	Power Automate Scheduled Flow	Runs daily at 08:00 AM automatically. Flow owned by Vanka Manjunath, all team members listed as co-owners.

Dashboard	Microsoft Power BI Desktop	5-tab governance dashboard. Free desktop version, connects directly to JSON output files.
Output format	JSON with provenance envelope	Machine-readable, schema-versioned, and easy for ShadowSight to ingest.
Run history	Excel via openpyxl	extraction_history.xlsx gives a human-readable record of every past run.
Identity service	Azure Entra ID	Microsoft's identity platform for app registration and permission management.
Runtime environment	Python 3.11 on Linux	Runtime used for the extraction engine. All five extractor modules and the orchestrator run on Python 3.11.
Resource Group	project-sentinel-rg	Azure resource group containing the app registration and related resources.
Output storage	/output/ directory	Output files written to the /output/ directory after each run. extraction_history.xlsx updated automatically.

2.3 Design Decisions

2.3.1 Python rather than PowerShell

We chose Python 3.11 as the extraction language rather than PowerShell. Microsoft provides PowerShell modules for Microsoft 365, but those are designed for interactive use and would have made it harder to build a non-interactive scheduled job. Python gave us better control over the full data pipeline authentication, API calls, pagination, analysis, normalisation, and export all in the same codebase. The team also had stronger Python skills, which meant fewer issues during Sprint 3 when we were building the five extraction modules.

2.3.2 MSAL Client Credentials for non-interactive authentication

The extraction job runs daily at 8am with nobody logged in. We needed an authentication method that works completely without a user. We used MSAL's Client Credentials flow, which lets the app authenticate directly to Azure Entra ID using a client ID and a client secret stored in config.py. The app is registered in the Sentinals.onmicrosoft.com tenant and the tenant admin granted consent for all the API permissions the tool needs. The token is valid for one hour, which is enough time for a full extraction run. If the token is close to expiry mid-run, the AuthModule requests a fresh one automatically.

2.3.3 BaseExtractor abstract class

Rather than writing five separate scripts each containing their own copy of the API call logic and JSON export code, we built a BaseExtractor abstract class and made all five extractor modules inherit from it. BaseExtractor provides shared logic: API client setup, following @odata.nextLink for pagination, ThrottleHandler for 429 responses, and the JSON export call. Each child class only needs to implement extract() to define which endpoint to call, and analyse() to define which risk checks to run. When the orchestrator calls run() on any extractor, it automatically calls extract() then analyse() then exports the result. This kept the codebase clean and made it easy to add a new extractor without touching existing code.

2.3.4 Output schema with provenance envelope (version 2.0.0)

Every JSON output file from Project Sentinel includes a provenance envelope a metadata block at the top of the file that records when the extraction ran, which tenant it came from, which schema version is in use, how many records came back, and whether API throttling was hit during the run. This block lets ShadowSight validate an incoming file before processing it. The schema version 2.0.0 with 95 fields across 9 categories.

2.3.5 Power Automate for scheduling

Instead of an Azure Function App, the team used Microsoft Power Automate to schedule and trigger the extraction pipeline. The flow named Project-Sentinel-Daily-Extraction runs on a daily schedule at 08:00 AM and is owned by Vanka Manjunath. This approach removed the need for a separate Azure hosting plan and kept the scheduling simple - all team members are listed as co-owners on the flow. The trade-off is that the tool depends on the Power Automate licence tied to the La Trobe tenant, which would need to be replaced with a different trigger mechanism in a production deployment.

2.4 Design Discussion

Scalability

The system was designed to handle more data as the tenant grows. The extraction engine follows pagination on every API endpoint it keeps requesting the next page of results until there are no more, so it can handle any number of records without running out. The Graph API allows up to 10,000 requests per 10-minute window, and the ThrottleHandler automatically waits and retries if that limit is hit. The scheduling is handled by Power Automate which runs the flow daily and can be reconfigured easily if the extraction needs to run more frequently.

Flexibility

The BaseExtractor design makes it straightforward to add new data sources. To add a new extractor for example, one that pulls Teams message metadata or Exchange audit events a developer just creates a new Python file in the extractors/ folder, inherits from BaseExtractor, and implements the two methods. The main orchestrator in main.py automatically picks it up. The output schema is also designed to be extended: new fields can be added to new categories without breaking the existing 95 fields. The JSON format means the output works with any downstream platform, not just ShadowSight.

Connection

Project Sentinel connects to four separate Microsoft API surfaces: Microsoft Graph v1.0 for most data, the Graph beta endpoint for sensitivity labels, the SharePoint REST API for site enumeration, and the Purview Compliance API for DLP data. On the output side, the JSON files are designed to be picked up by ShadowSight via HTTPS or SFTP the provenance envelope at the top of each file gives ShadowSight everything it needs to validate the file before importing it. The Power BI dashboard connects directly to the JSON output files, so there is no separate database or data warehouse needed. If SECMON1 wanted to send the output somewhere else for example, a SIEM or a logging platform the JSON format makes that easy to do.

Usability

For a security analyst, using the tool is simple. The Power Automate flow runs automatically every day at 8am, so in most cases the analyst does not need to do anything the new extraction data is just there. If they need to trigger a manual run, they run one command: python main.py. The extraction_history.xlsx file gives a plain summary of every past run with the record counts and data quality score. The Power BI dashboard has five clearly labelled tabs that cover the main governance questions: how many files, who has access, what is the risk level, and which users are most active. We also ran a usability trial in Sprint 3 to check that the documentation was clear enough for someone who had not been involved in building the tool most participants completed the setup steps without needing help.

Security

The app registration uses only read-only permissions the tool cannot write, modify, or delete anything in the Microsoft 365 tenant. All API calls go to Microsoft's own endpoints over HTTPS, so no data passes through any third-party service. The output JSON files are stored in the configured output directory after each extraction run and can be used by Power BI or ShadowSight.t until ShadowSight collects them. The main security concern we identified is that the client secret is currently stored in config.py as a plain text value, which is fine for a development environment but not suitable for a production deployment.

Before going live, the client secret should be moved to Azure Key Vault and accessed securely by the extraction environment using managed identity or an equivalent secure mechanism. No user data, tenant data, or credentials were stored outside of the Azure environment at any point during development or testing.

2.5 Known Limitations

Retention and records management fields: The retentionRecords category is fully defined in the schema and the PurviewLabelExtractor module is built for it, but these fields could not be populated during live testing. Populating them requires specific Microsoft Purview configuration that was not set up in the Sentinals.onmicrosoft.com developer environment. These fields currently return null in all output files.

DLP extraction requires E5 licensing: The DLPExtractor module and the dlpMatches fields in the schema need a Microsoft 365 E5 license or the Microsoft Compliance add-on. The developer tenant operates on E5, so DLP extraction returns empty arrays. The code is built and ready it just needs an E5 tenant to work against.

Power BI dashboard refresh is manual only: There is no automatic refresh on the dashboard. We planned to set up a Power BI gateway that would trigger a refresh after each Monday extraction, but this was not completed during the project. A security analyst must open Power BI Desktop and click Refresh manually to see updated extraction data.

Single developer tenant: The tool has been built and tested against one developer tenant only (Sentinals.onmicrosoft.com). Behaviour against larger production tenants particularly pagination volume and API response shapes has not been tested.

2.6 Planned Improvements

These are things the team identified during the project that would make the tool better, but could not be completed within the six-sprint timeline.

Table 2.6 Planned Improvements

Improvement	Why It Would Help	What Is Needed
Move client secret to Azure Key Vault	Removes the security risk of storing credentials in config.py	Azure Key Vault resource and secure secret retrieval mechanism for the extraction environment.
Automatic Power BI refresh after Monday extraction	Analysts would see updated data without any manual steps	Power BI gateway configured and connected to the Project Sentinel output directory.
Upgrade developer tenant to E5 licensing	Enables DLP extraction and eDiscovery fields that currently return null	Microsoft 365 E5 developer subscription or compliance add-on

Add automated test suite	Would catch breaking changes in the Graph API automatically	pytest test cases for each extractor module + GitHub Actions CI/CD
Expand GitHub repository documentation	Helps future developers understand and extend the tool faster	Add detailed README, inline code comments, and contribution guide to github.com/YaswanthKanderi/Project-Sentinel
Add a 6th extractor for Teams message metadata	Would give SECMON1 visibility into Teams channel activity	Teams.ReadBasic.All permission + new TeamsMessageExtractor class
Multi-tenant support	Would allow SECMON1 to run extractions across multiple client tenants	Config file redesign to support multiple tenant configurations

3. Reference and Vendor Documentation

3.1 Reference Documents

The following documents were used by the team to guide the design, development, and testing of Project Sentinel. All of these documents are either in the IDP project folder or linked below.

Table 3.1 Reference Documents

Document	Author / Source	Purpose
Project Brief - Project Sentinel - SECMON1	Christopher McNaughton, SECMON1	Original project requirements and must-have deliverables
Output Schema Specification (v2.0.0)	Lahari Borra	Full 95-field schema definition - 9 categories, data types, validation rules
Governance Data Identification	Yaswanth Kanderi	SharePoint metadata fields and governance attributes mapped to API sources
Microsoft 365 Non-Interactive Authentication Configuration	Raj Kumar Mustyala	Authentication design - Client Credentials flow, token endpoint, required params
MS365 Developer Tenant Setup Guide	Yaswanth Kanderi	Step-by-step guide for provisioning and configuring the test tenant
Sprint 3 Sample Data Extraction and Analysis	Vanka Manjunath	Manual extraction results, permission analysis, risk scoring framework
Sentinel Usability Trial Presentation	Lahari Borra	Usability trial methodology, participant results, and improvements made
Data Validation Rules	Vanka Manjunath	Rules used to calculate dataQualityScore for each extraction run.

3.2 Existing System Documentation

The following items form the existing system documentation that a future developer or maintainer should refer to when working with Project Sentinel.

The handover_backend/ folder (unpacked from Files_Backend.zip in the IDP folder) contains the full Python source code including main.py, config.py, the five extractor modules, and requirements.txt. This is the primary codebase reference.

The Output_Schema_Specification.json file is the complete machine-readable schema definition. Every output JSON file the tool produces must match this schema.

The extraction_history.xlsx file in the output folder is updated after every run. It gives a plain history of all past extractions - timestamps, record counts, quality scores, and any errors.

The GitHub repository contains the full source code with version history. The repository was set up and maintained by Yaswanth Kanderi and is available at: https://github.com/YaswanthKanderi/ProjectSentinel

3.3 Vendor and Tool Documentation

Below are the official documentation links for every external tool, API, and library used in this project. These are the first places to look when something breaks or needs to be updated.

Microsoft Authentication and Identity

MSAL for Python token requests and caching: https://learn.microsoft.com/en-us/entra/msal/python/

Azure Entra ID App Registration how to set up the app: https://learn.microsoft.com/en-us/entra/identity-platform/quickstart-register-app

OAuth 2.0 Client Credentials Flow the auth method the tool uses: https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow

Microsoft Graph API

Graph API overview and getting started: https://learn.microsoft.com/en-us/graph/api/overview

Graph Explorer test API calls in the browser without code: https://developer.microsoft.com/en-us/graph/graph-explorer

Graph API changelog check for breaking changes to endpoints: https://learn.microsoft.com/en-us/graph/changelog

SharePoint and Purview

SharePoint REST API documentation: https://learn.microsoft.com/en-us/sharepoint/dev/sp-add-ins/get-to-know-the-sharepoint-rest-service

Microsoft Purview documentation sensitivity labels, DLP, retention: https://learn.microsoft.com/en-us/purview/

Power Automate and Deployment

Power Automate documentation: https://learn.microsoft.com/en-us/power-automate/

Create a scheduled flow in Power Automate: https://learn.microsoft.com/en-us/power-automate/run-scheduled-tasks

Azure Key Vault - for storing secrets securely in production: https://learn.microsoft.com/en-us/azure/key-vault/general/overview

Python Libraries

requests library documentation: https://docs.python-requests.org/en/latest/

openpyxl documentation used to write extraction_history.xlsx: https://openpyxl.readthedocs.io/en/stable/

Dashboard

Power BI Desktop download (paid): https://powerbi.microsoft.com/en-us/downloads/

Power BI documentation: https://learn.microsoft.com/en-us/power-bi/

Development Environment

Microsoft 365 Developer Program - set up a free E5 test tenant: https://developer.microsoft.com/en-us/microsoft-365/dev-program

3.4 Minimum System Requirements

These are the minimum requirements to run Project Sentinel locally or deploy it to Azure. These specs are based on what the team used during development and testing.

Table 3.4 Minimum System Requirements

Component	Minimum Requirement
Operating System	Windows 10+, macOS 12+, or Ubuntu 20.04+
Python Version	Python 3.11 or later (used by the extraction engine during execution)
RAM	4 GB minimum - 8 GB recommended for larger extraction runs
Disk Space	256 GB minimum – 512 GB maximum for larger extensions
Internet Access	Stable broadband connection - required for all Microsoft API calls
Azure Account	Azure subscription with permission to create App Registrations and manage Azure resources.
Microsoft 365 Tenant	Developer tenant or production MS365 tenant - admin consent required for all permissions
Power BI Desktop	Paid version - needed to open and refresh the governance dashboard

Text Editor or IDE	Any - VS Code recommended for editing Python files and config

3.5 Dependency and License Table

The table below lists all Python packages the tool depends on, along with their open-source license type and any known security notes. All four packages are actively maintained and have no known active vulnerabilities as of May 2026. Run pip list - outdated regularly to check for updates.

Table 3.5 Python Package Dependencies, Licenses, and CVE Notes

Package	Version	License	Purpose	Security Notes / CVEs
msal	>=1.24.0	MIT	OAuth 2.0 Client Credentials token requests and caching	No active CVEs. Microsoft actively maintains this library. Keep updated to get the latest token handling fixes.
requests	>=2.31.0	Apache 2.0	HTTP client for all Microsoft Graph and SharePoint API calls	Older versions (below 2.28) had known CVEs. Always stay on 2.31 or higher. Run pip install requests upgrade to update.
openpyxl	>=3.1.2	MIT	Writes extraction_history.xlsx after each run	No known active CVEs. Straightforward Excel writing library with no external dependencies.
python-dateutil	>=2.8.2	Apache 2.0	Parses and compares ISO 8601 timestamps in API responses	No known active CVEs. Widely used and stable. No changes needed unless Microsoft changes timestamp formats.

To check for vulnerabilities in any of the installed packages, run:

```
pip install pip-audit
pip-audit
```

This scans all installed packages against the Python vulnerability database and reports anything that needs updating.

4. APIs, Tools and Dependencies

4.1 Microsoft APIs Used

The tool makes calls to four Microsoft API surfaces. The stable v1.0 Graph API handles most requests. Sensitivity labels and retention labels require the beta endpoint because Microsoft has not yet promoted those operations to v1.0.

Table 4.1 API Endpoints

API	Endpoint	Purpose	Module
Graph v1.0	GET /auditLogs/signIns	Sign-in events for all users	AuditLogExtractor
Graph v1.0	GET /security/auditLog/queries	Unified directory audit log	AuditLogExtractor
Graph v1.0	GET /drives/{id}/items	File and folder metadata	FileMetadataExtractor
Graph v1.0	GET /sites/{id}/lists	SharePoint site and library list	FileMetadataExtractor
Graph v1.0	GET /drives/{id}/items/{id}/permissions	Sharing permissions per file	SharingPermissionsExtractor
Graph v1.0	GET /drives/{id}/items/{id}/versions	Version history per file	FileMetadataExtractor
Graph beta	GET /drives/{id}/items/{id}/extractSensitivityLabels	Purview sensitivity labels	PurviewLabelExtractor
Graph beta	GET /drives/{id}/items/{id}/retentionLabel	Retention label status	PurviewLabelExtractor
Purview	GET /security/informationProtection/sensitiveTypes	DLP policy matches	DLPExtractor

4.2 Required API Permissions

All permissions are Application type the app authenticates using its own identity via Client Credentials, not on behalf of any user. All permissions required tenant admin consent before the first extraction could run. The team reviewed all permissions against a least-privilege principle every permission in the list below is the minimum required to access the data that module needs.

Table 4.2 Microsoft Graph API Permissions

Permission	Purpose	License Requirement
Sites.Read.All	Read SharePoint site collections and document libraries	MS365 E5
Files.Read.All	Read all files in SharePoint and OneDrive	MS365 E5
AuditLog.Read.All	Read the Microsoft 365 unified audit log	MS365 E5
InformationProtectionPolicy.Read.All	Read sensitivity labels from Microsoft Purview	MS365 E5
RecordsManagement.Read.All	Read retention labels and records management status	MS365 E5
User.Read.All	Resolve user IDs to display names, check account status	MS365 E5
Directory.Read.All	Read users, groups, and organisational structure	MS365 E5
eDiscovery.Read.All	Read legal hold status from eDiscovery cases	MS365 E5 required
DataLossPreventionPolicy.Evaluate.All	Read DLP policy matches on files	MS365 E5 required

4.3 Python Dependencies

The four packages in requirements.txt cover everything the extraction engine needs to run. Full license and security details for each package are in Section 3.5.

Table 4.3 Python Package Dependencies

Package	Version	Purpose
msal	>=1.24.0	Microsoft Authentication Library handles OAuth 2.0 Client Credentials token requests
requests	>=2.31.0	HTTP client used for all Microsoft API calls

openpyxl	>=3.1.2	Writes extraction_history.xlsx after each run
python-dateutil	>=2.8.2	Parses and compares ISO 8601 timestamps in API responses

4.4 Code Structure

The full backend codebase is in the handover_backend folder (unpacked from Files_Backend.zip in the IDP folder). This is how the files are laid out:

Repository Structure

```
Project-Sentinel/
├── main.py
├── config_template.py
├── host.json
├── requirements.txt
├── README.md
├── auth/
│   ├── __init__.py
│   └── msal_auth.py
├── extractors/
│   ├── __init__.py
│   ├── audit_logs.py
│   ├── dlp_extractor.py
│   ├── file_metadata.py
│   ├── purview_labels.py
│   └── sharing_permissions.py
└── utils/
    ├── __init__.py
    ├── excel_exporter.py
    ├── exporter.py
    ├── logger.py
    ├── throttle_handler.py
    └── timeline.py
```

5. Deployment and Operations Guide

5.1 Azure Infrastructure Overview

The deployment consists of three connected components that must be configured before Project Sentinel can operate successfully. The first is the Azure Entra ID App Registration, which handles authentication and grants the required permissions to access Microsoft 365 APIs. The second is the Microsoft Power Automate Scheduled Flow, which automatically triggers the extraction process each day. The third is the output storage location, where all JSON output files and the extraction history log are saved after each run.

These components operate within the Sentinals.onmicrosoft.com developer tenant and work together to provide a fully automated extraction process. If any component is missing or incorrectly configured, the extraction workflow may fail. The following sections describe each component and its role in the system architecture.

5.1.1 Azure Entra ID App Registration

The app is registered in the Sentinals.onmicrosoft.com tenant. It uses a client secret for authentication. Before the tool can make any API calls, the tenant admin must grant admin consent for all the permissions listed in Section 4.2. The registration produces the three values needed for config.py: Tenant ID, Client ID, and Client Secret.

The app registration details are: Display name: project-sentinel, Application (client) ID: 46adb1d8-2952- 44e9-9b17-4321859e4a1e, Directory (tenant) ID: 5c2ec200-752e-42c9-bb73-e0333eeffaf7. The registration was created on 14 May 2026 and has 1 active client secret with no certificates.

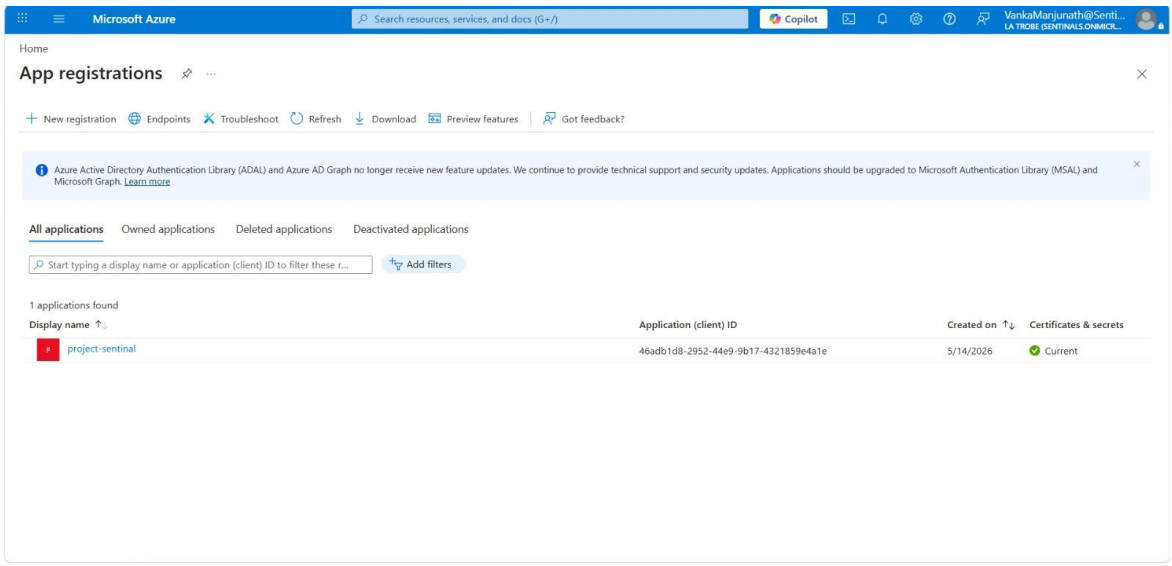

Figure 5.1.1 Azure Entra ID App Registration: project-sentinel

5.1.2 Power Automate Scheduled Flow

Instead of an Azure Function App, the team used Microsoft Power Automate to schedule and trigger the extraction runs. The flow is named Project-Sentinel-Daily-Extraction and is owned by Vanka Manjunath (VankaManjunath@Sentinals.onmicrosoft.com). It was created on May 21, 2026 and is connected to Microsoft Teams.

The flow type is Scheduled and runs daily at approximately 08:00 AM. As of May 30, 2026, all runs show a status of Succeeded in the 28-day run history. The flow triggers the extraction pipeline automatically each day. All team members (VM, R, Y, JP, LB) are listed as co-owners on the flow. The flow runs on the owner plan within the La Trobe (default) Power Automate environment.

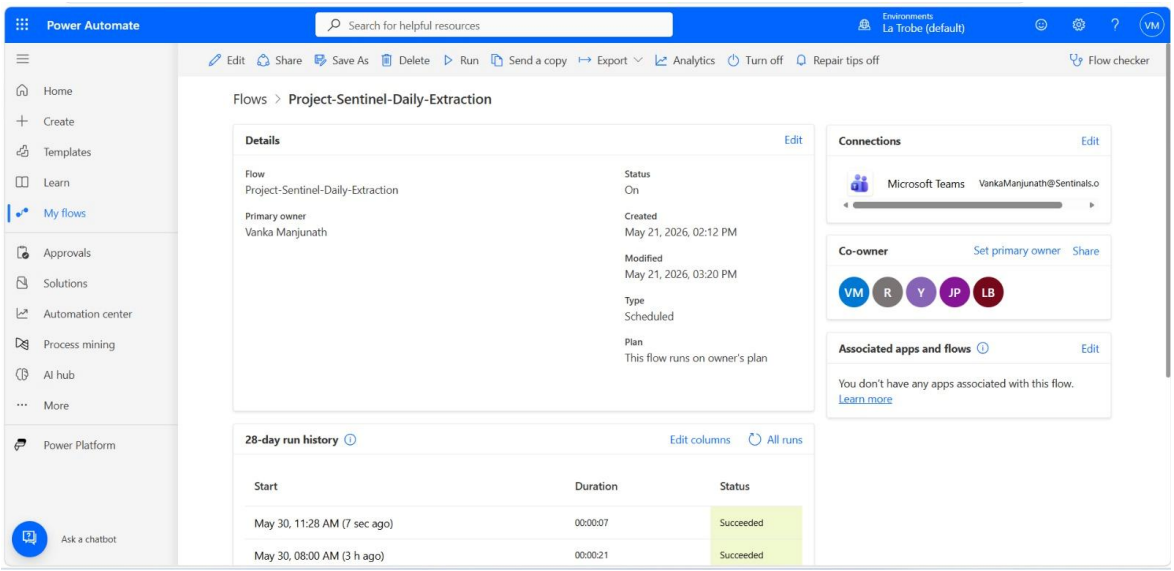

Figure 5.1.2 Power Automate Scheduled Flow: Project-Sentinel-Daily-Extraction

5.1.3 Output Storage

Each run writes nine JSON output files to the /output/ directory. Every file name includes a timestamp so runs never overwrite each other. The nine files cover audit logs, sign-in summaries, file metadata, file metadata summaries, sharing permissions, sharing summaries, Purview label data, DLP data, and a full governance report.

The extraction_history.xlsx file in the same directory keeps a running log of every past run. A new row is added at the end of each successful extraction recording the timestamp, record counts for each module, the overall dataQualityScore, and any errors that occurred. This file is the quickest way to check whether the tool has been running correctly and to compare results across different runs.

If the /output/ directory does not exist when the tool runs, it is created automatically by the exporter module. No manual setup of the output folder is needed.

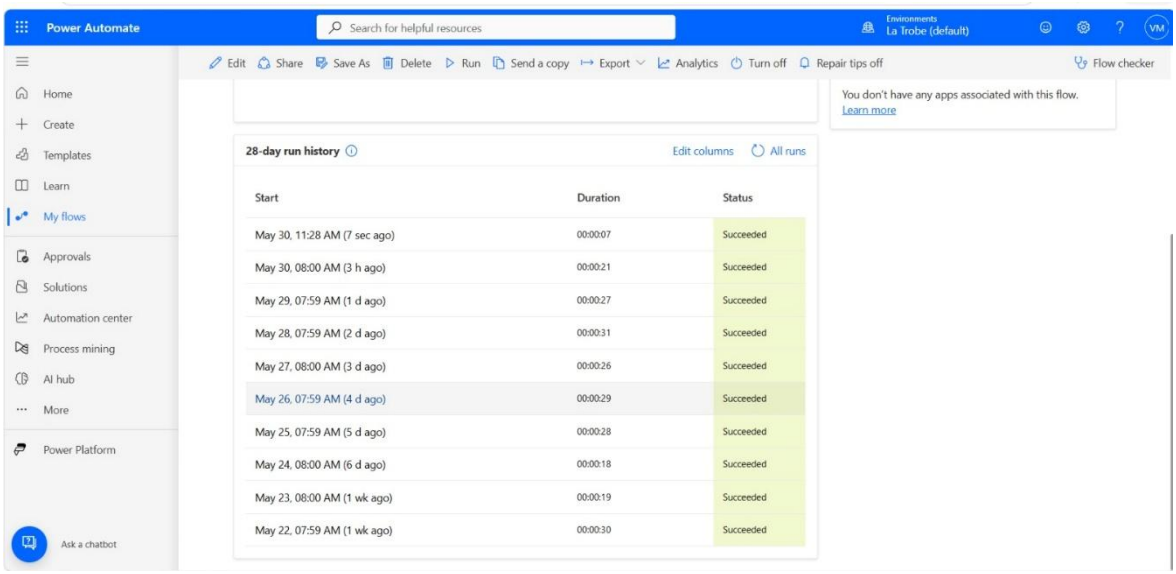

Figure 5.1.3 Power Automate Scheduled Flow : 28-day run history

5.2 Running the Tool Manually

The tool normally runs automatically every day through the Power Automate scheduled flow. However, if you need to trigger a one-off extraction for example, after a code change, to test a fix, or to re-run a failed extraction you can run it manually from the command line.

Before running, make sure config.py has the correct Tenant ID, Client ID, Client Secret, and Tenant Domain values. Also make sure you have Python 3.11 installed and all dependencies from requirements.txt are in place. Then run the following three commands in order:

```
cd handover_backend
pip install -r requirements.txt
python main.py
```

main.py is the orchestrator. It creates an ExtractionJob, runs all five extractor modules in sequence, runs the governance analysis, and writes all output files to the /output/ directory. A full run typically takes between two and five minutes depending on the size of the tenant and whether API throttling is hit.

While the tool is running you will see log output in the terminal for each module as it completes. If a module fails, the error will be printed with the module name and the reason. The run will continue with the remaining modules even if one fails. After the run finishes, check the /output/ folder for the new JSON files and open extraction_history.xlsx to confirm a new row was added with the correct record counts.

5.3 Configuration

All configuration values go into config.py before the tool will run:

```
TENANT_ID     = "your-tenant-id"
CLIENT_ID     = "your-app-client-id"
```

```
CLIENT_SECRET = "your-app-client-secret"
TENANT_DOMAIN = "Sentinals.onmicrosoft.com"
```

Do not commit config.py with real credentials to any version control system. For a production deployment, the client secret should be stored in Azure Key Vault and accessed via a managed identity rather than a config file.

The token endpoint for the Client Credentials auth flow is:

```
POST https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/token
```

Required parameters: grant_type = client_credentials, client_id, client_secret, scope = https://graph.microsoft.com/.default

5.4 Output Files

Each successful extraction run produces up to nine output files. The TIMESTAMP in each filename is formatted as YYYYMMDD_HHMMSS in UTC. Using timestamps in the filenames means no two runs ever overwrite each other - every run leaves a permanent record in the /output/ folder.

The nine files are split into detail files and summary files. The detail files contain the full raw records extracted from the MS365 APIs. The summary files contain calculated statistics and aggregated counts from the same data. Both types follow the Output Schema v2.0.0 format and include a provenance envelope at the top so downstream systems like ShadowSight can validate the source and timestamp before importing.

The DLP output file (dlp_matches_TIMESTAMP.json) is created on every run but will contain no API-level policy match records in the current E5 developer tenant - full DLP API extraction requires MS365 E5 licensing. However, the Sentinel-Teams-DLP-Policy configured in Microsoft Purview operates independently of the extraction API and actively monitors the tenant in simulation mode. Alerts from this policy are delivered via email when sensitive information types are detected. All other eight JSON output files are fully populated on every run.

Table 5.4 Output Files per Extraction Run

File	Contents
audit_logs_signin_TIMESTAMP.json	All sign-in audit log records for the extraction period
audit_logs_signin_SUMMARY_TIMESTAMP.json	Summary statistics: total sign-ins, failures, risk levels, top users
file_metadata_TIMESTAMP.json	All file metadata records from SharePoint drives
file_metadata_summary_SUMMARY_TIMESTAMP.json	Summary: total files, sharing status, sensitivity label coverage

sharing_permissions_TIMESTAMP.json	All sharing permission records per file
sharing_permissions_SUMMARY_TIMESTAMP.json	Summary: total permissions, anonymous link count, external user count
sensitivity_labels_TIMESTAMP.json	Sensitivity label data (requires Purview configured in tenant)
dlp_matches_TIMESTAMP.json	DLP policy match data (API extraction requires MS365 E5 licensing - Sentinel-Teams-DLP-Policy monitors via Purview independently)
extraction_history.xlsx	Cumulative run log all past extractions with record counts and status

5.5 Updating and Patching

This section covers the three most common maintenance tasks updating the source code, upgrading Python packages, and rotating the client secret. All three should be done carefully because a mistake in any one of them will stop the tool from running.

The client secret in the app registration was created on 14 May 2026 and has an expiry date. When the secret gets close to expiry, a new one must be generated in the Azure portal under Certificates and Secrets, and the new value must be updated in the value in config.py or the secure configuration location used by the deployment environment.

If the secret expires before it is rotated, all extraction runs will fail with an authentication error until a new secret is in place. Follow the steps below for any of these three scenarios:

Table 5.5 Update and Patching Steps

Step	Action	Details
1	Pull the latest code from GitHub	Run: git pull origin main. This brings in any bug fixes or new features.
2	Update Python packages	Run: pip install -r requirements.txt - upgrade. This gets the latest versions of msal, requests, openpyxl, and python-dateutil.
3	Check for new CVEs	Run: pip-audit. These scans installed packages against the Python vulnerability database and flags anything that needs patching.
4	Rotate the client secret if needed	Client secrets expire on the date set when created. In the Azure portal, go to the App Registration, then Certificates & Secrets, generate a new secret, and update the value in config.py or the secure configuration location used by the deployment environment.

5	Check the Graph API changelog	Visit https://learn.microsoft.com/en-us/graph/changelog to check for any breaking changes to the endpoints the tool uses.
6	Test locally before redeploying	Run: python main.py. Check that all 9 output files are created and that extraction_history.xlsx has a new row.
7	Redeploy to Azure	Update the deployment environment and verify that the Power Automate scheduled flow can successfully trigger the latest version of the extraction engine.

5.6 Extending the System

One of the main reasons we built the BaseExtractor class was to make it easy to add new data sources without having to change existing code. Here is exactly how to add a new extractor module for example, one that pulls Teams message metadata.

Table 5.6 Steps to Add a New Extractor Module

Step	What to Do
1	Create a new file in the extractors/ folder. Name it after what it extracts for example, teams_messages.py.
2	At the top of the file, import BaseExtractor: from extractors.base import BaseExtractor
3	Define your new class inheriting from BaseExtractor: class TeamsMessageExtractor(BaseExtractor):
4	Write the extract() method. This is where you put your Graph API endpoint and the pagination loop. Copy the pattern from an existing extractor like audit_logs.py.
5	Write the analyse() method. This is where you add your risk checks for the new data for example, flagging messages that contain sensitive keywords.
6	Open main.py and import your new class: from extractors.teams_messages import TeamsMessageExtractor
7	Add your new extractor to the ExtractionJob list so it runs alongside the existing five modules.
8	Add any new API permission (e.g., Teams.ReadBasic.All) to the app registration in Azure Entra ID and grant admin consent.
9	Test it locally: python main.py. Check that a new output file appears in /output/ and that extraction_history.xlsx shows the new module.

10	Update requirements.txt if you added any new Python packages, then deploy the updated code to the extraction environment and verify that the Power Automate scheduled flow can successfully execute the new version.

5.7 Monitoring and Troubleshooting

To check when the last extraction ran and how many records it returned, open extraction_history.xlsx. Each row represents one run with the timestamp, record counts per module, errors, and the data quality score.

If a run fails, the error is written to the extraction logs generated during execution. Common causes: an expired client secret (secrets have a set expiry date configured in Azure Entra ID), loss of API permission consent (can happen if the tenant admin revokes consent), or a Microsoft API outage. Execution status can be monitored through the Power Automate run history. Any extraction errors are recorded in the execution logs and extraction_history.xlsx for troubleshooting purposes.

Table 5.7 Common Errors and Fixes

Error	Likely Cause	Fix
AADSTS7000215: Invalid client secret	Client secret has expired	Generate a new secret in Azure portal App Registration and update config.py or Application Settings
HTTP 403 Forbidden on API call	Admin consent was revoked or a permission is missing	Re-grant admin consent in Azure portal under API Permissions
HTTP 429 Too Many Requests (not auto-resolved)	Throttling exceeded 5 retries	Wait a few minutes and re-run. Check extraction_history.xlsx for the Retry-After values that were logged.
output/ folder empty after run	Run failed silently or config.py has wrong values	Check the execution logs and Power Automate run history. Verify all four config values are correct.
dataQualityScore below 70	Optional fields returning null (normal for E5 tenant)	Expected behaviour - DLP and retention fields return null without E5 licensing. Score will be 78+ for a healthy run.

5.8 Learning Resources

If you are new to any of the technologies used in this project, the links below are the best starting points. These are the same resources the team used during development.

Microsoft 365 and Graph API

Microsoft Graph API overview: https://learn.microsoft.com/en-us/graph/api/overview

Graph Explorer (test API calls without code): https://developer.microsoft.com/en-us/graph/graph-explorer

MSAL Python library documentation: https://learn.microsoft.com/en-us/entra/msal/python/

OAuth 2.0 Client Credentials flow explained: https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow

Power Automate and Deployment

Power Automate getting started: https://learn.microsoft.com/en-us/power-automate/

Scheduled flows in Power Automate: https://learn.microsoft.com/en-us/power-automate/run-scheduled-tasks

Azure Key Vault for secrets management: https://learn.microsoft.com/en-us/azure/key-vault/general/overview

Domain Knowledge - Microsoft 365 Governance

Microsoft 365 governance refers to the set of policies, controls, and monitoring practices that organisations use to manage how their data is accessed, shared, and stored across Microsoft 365 services. Key concepts for understanding Project Sentinel:

audiaudir labels are tags applied to files in Microsoft Purview for example, Public, Internal, Confidential, or Highly Confidential. They define what protections apply to the file. If a file has no sensitivity label, it means no classification has been applied, which is a governance risk.

Data Loss Prevention (DLP) policies detect when files contain sensitive information types for example, Australian Tax File Numbers or credit card numbers. When a match is found, the policy can block sharing, alert the security team, or log the event.

Sharing permissions in SharePoint come in three types: inherited (the file takes access from its parent folder), direct (a specific user or group is granted access to just this file), and link-based (anyone with the link can access it). Anonymous links where anyone with the URL can view the file without signing in are the highest risk type.

Microsoft Purview information protection overview: https://learn.microsoft.com/en-us/purview/information-protection

Microsoft 365 Developer Program (set up a free test tenant): https://developer.microsoft.com/en-us/microsoft-365/dev-program

6. Testing and Validation

6.1 Live Extraction Tests

We ran two live extraction tests - one in Sprint 4 and one in Sprint 5 - against the real Sentinals.onmicrosoft.com developer tenant. Both used the actual extraction engine and pulled real data from SharePoint, sign-in logs, and sharing permissions. The numbers below are what the tool actually returned.

Table 6.1 Live Extraction Results

Metric	Sprint 4 (14 May 2026)	Sprint 5 (May 2026)
Total sign-in records extracted	183	354
Successful sign-ins	165	302
Failed sign-ins	18	52 (14.7% of total)
Risky sign-ins detected	0	0
File metadata records extracted	18	26
Files with no sensitivity label applied	18 out of 18 (100%)	26 out of 26 (100%)
Files shared without private access boundary	18 out of 18 (100%)	26 out of 26 (100%)
Sharing permission records extracted	72	105
Anonymous sharing links detected	0	0
External users with file access	0	0
API throttling (HTTP 429) encountered	Yes- handled correctly	Yes- handled correctly
Output files produced	9 of 9	9 of 9

Both runs completed without any records being lost. The ThrottleHandler correctly managed rate limiting in both sprints. The key finding from both runs is that every file in the tenant is shared without restriction and none have sensitivity labels applied. HR, Finance, and Legal folders contain files that all Members and Visitors groups can access with no separation flagged as HIGH RISK in the Sprint 5 security analysis.

6.2 Risk Scoring Framework

During Sprint 3, the team built a risk scoring framework to give each extracted file a consistent, repeatable governance risk score. This framework was designed to feed into the ShadowSight platform's analytical layer. Each file gets a cumulative score based on the conditions present.

Table 6.2.1 Risk Scoring Rules

Risk Condition	Detection Logic	Score Added	Severity
Anonymous sharing link is active	isAnonymousLink = true	+40	**High**
Sharing link has no expiry date	sharingLinkExpiry = null	+25	**High**
Write access granted to an external user	isExternal = true + role = write	+20	**High**
External user has any level of access	isExternal = true	+15	**Medium**
Sensitivity label is missing	sensitivityLabelId = null	+15	**Medium**
No DLP policy match on a classified file	dlpPolicyName = null on classified file	+15	**Medium**
File has not been modified in over 180 days	inactivityDays > 180	+10	**Medium**
Version history is disabled on the library	versioningEnabled = false	+10	**Medium**
File owner account is disabled in Azure Entra ID	orphanedFile = true	+10	**Medium**
File has unique permissions (inheritance broken)	uniquePermissions = true	+5	**Low**

Table 6.2.2 Risk Score Bands and Actions

Score Range	Risk Level	Recommended Action
0 to 20	**Low**	No immediate action needed. File is within normal governance boundaries.
21 to 49	**Medium**	Review recommended. Flag for governance team awareness within 30 days.
50 and above	**High**	Immediate action required. Escalate to security team for investigation.

Example: A file with an anonymous link (score +40) and no expiry date (+25) would score 65 - flagged as High Risk. A file with only an external authenticated user with an expiry date set would score +15 - Low Risk. In the Sprint 3 manual extraction, 2 out of 10 files scored 65 (anonymous links with no expiry), and 2 scored 15 (external links with expiry dates set).

6.3 Security Risk Assessment

We reviewed how the tool handles credentials and data during extraction.

The client secret is currently stored as a plain text value in config.py. This is fine for the development environment but is not suitable for a real production deployment. Before going live, the secret should be moved to Azure Key Vault and accessed securely by the extraction environment rather than being stored directly in config.py.

No data leaves the Microsoft 365 tenant boundary during extraction. All API calls go to Microsoft endpoints only. The output JSON files remain within the Project Sentinel extraction environment until ShadowSight collects them. No third-party services or external APIs are called during any extraction run.

The app registration uses the minimum permissions needed - all are read-only Application type permissions. The app cannot write, modify, or delete any content in the tenant.

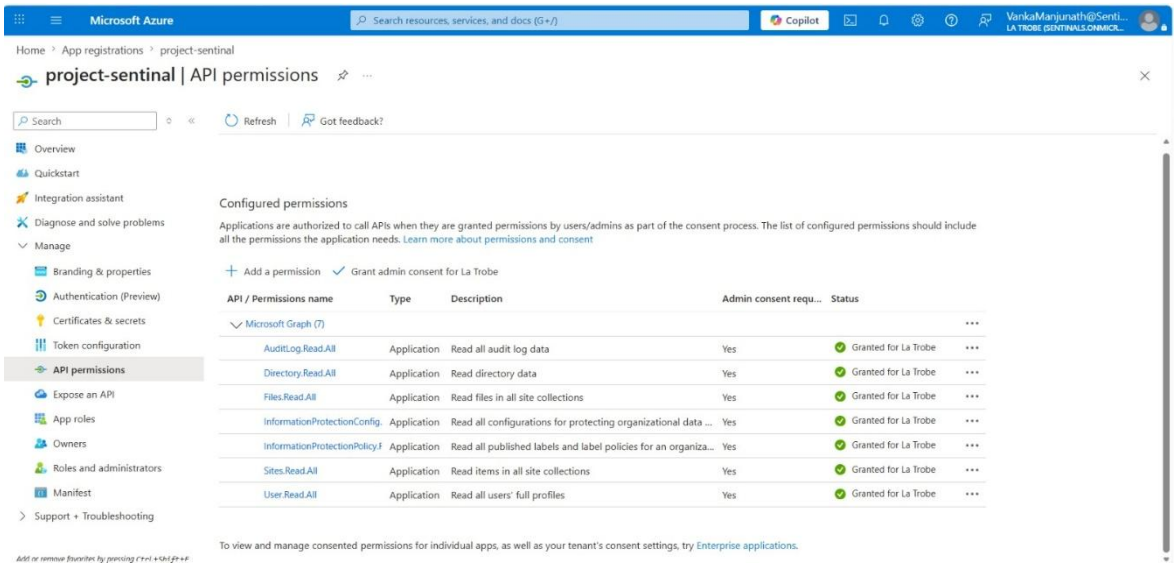

Figure 6.3 Azure App Registration: API Permissions

6.3.1 DLP Policy Configuration and Testing

A Microsoft Purview DLP policy named Sentinel-Teams-DLP-Policy was configured and activated in simulation mode during Sprint 5. The policy covers five locations - Exchange email, SharePoint sites, OneDrive accounts, Microsoft Teams chat and channel messages, and on-premises repositories. It contains three rules: Sentinel-Teams-Sensitive-Rule, a low volume detection rule, and a high-volume detection rule. Email notifications are enabled and alerts are sent automatically when a rule is triggered.

During testing on 31 May 2026, the policy successfully detected a document titled "DLP TEST FILE" in OneDrive for Business, authored by Yaswanth Kanderi (yaswanthkanderi@Sentinals.onmicrosoft.com). The document contained an Australia Tax File Number, which matched the Sentinel-SensitiveData-Rule condition. A low-severity alert was generated and delivered via Office365Alerts@microsoft.com to Vanka Manjunath, Yaswanth Kanderi, and Raj Kumar Mustyala. This confirms the DLP policy is correctly configured and actively monitoring the tenant for sensitive information types.

The policy is currently running in simulation mode, meaning it detects and alerts but does not block file sharing. Before moving to production, the policy should be switched to enforcement mode so that sharing of files containing sensitive information types can be actively restricted rather than only reported.

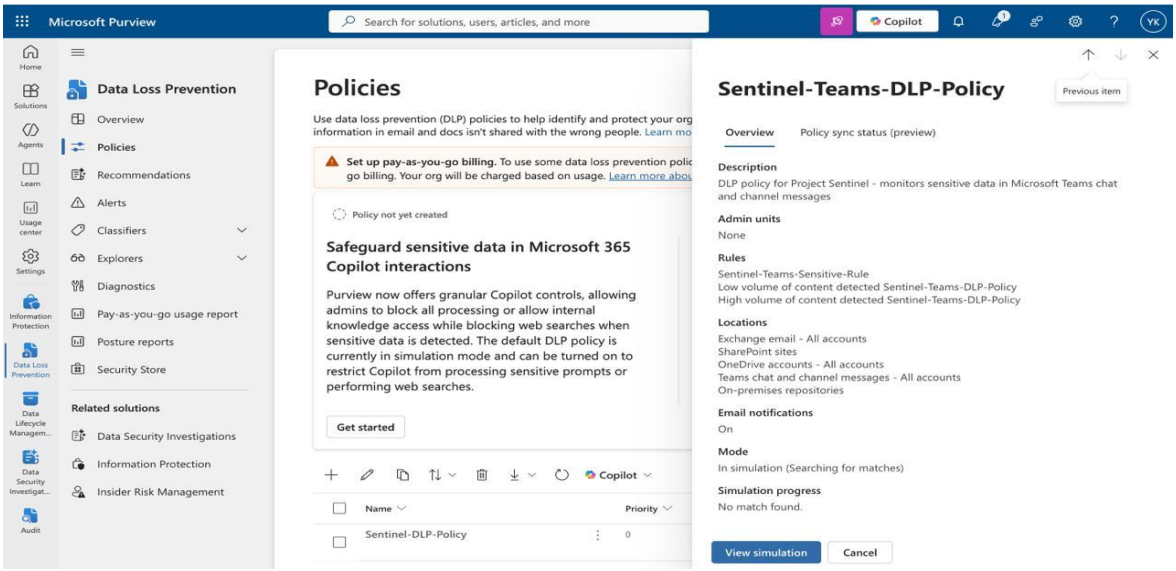

Figure 6.3.1 - Microsoft Purview DLP Policy: Sentinel-Teams-DLP-Policy

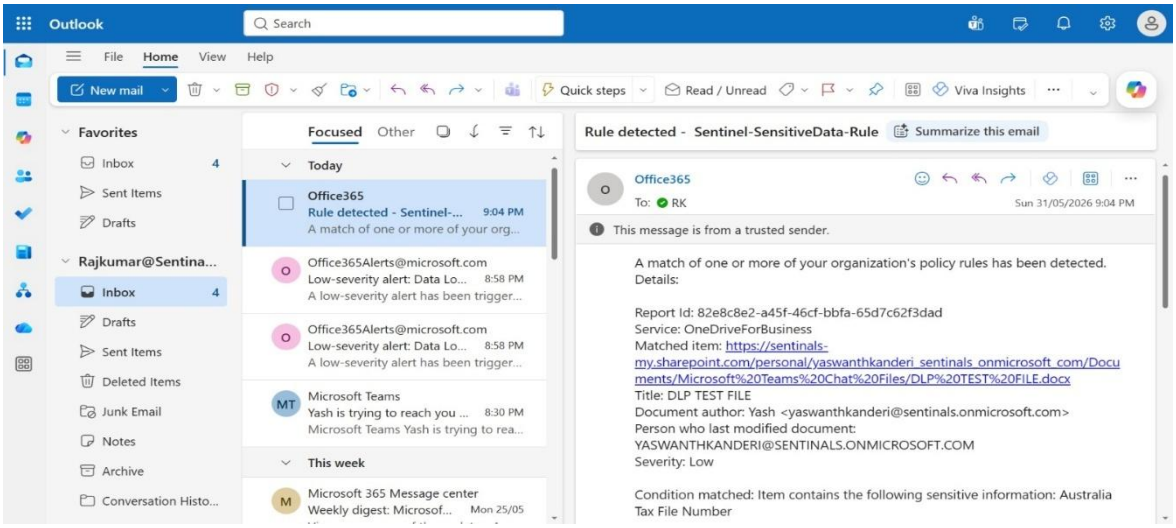

Figure 6.3.2 - DLP Alert Email: Sentinel-SensitiveData-Rule Triggered

6.4 Usability Trial

We ran a usability trial in Sprint 4 to check how easy the tool and its documentation are to follow. Participants were given the User Documentation and asked to work through setup tasks: configuring their environment, connecting to the Azure Entra ID app registration, running a test extraction, and interpreting the output files. Feedback was collected through a Google Form.

Most participants completed the authentication and extraction steps without needing help. The step participants found hardest was the Azure Entra ID configuration in the Azure portal - the screenshots in the User Documentation were updated after the trial to make those steps clearer.

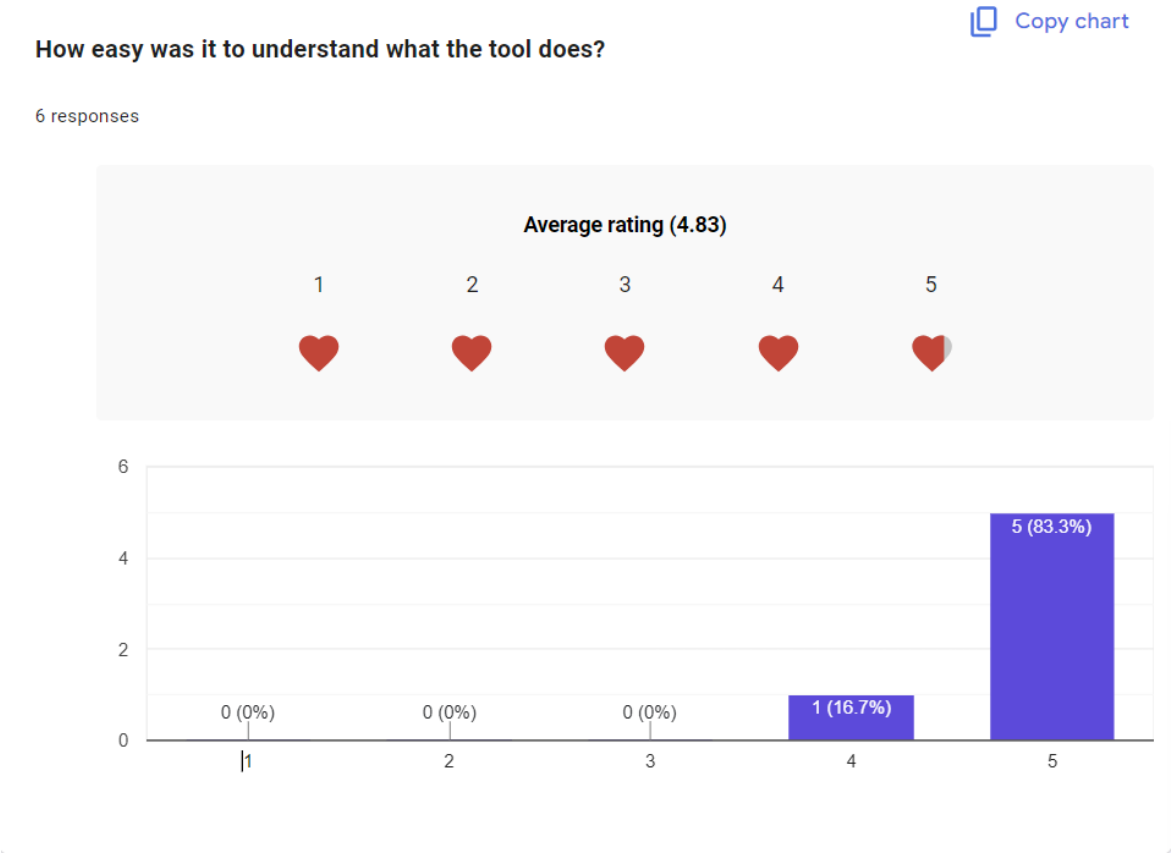

Figure 6.4.1 Usability Trail Q1

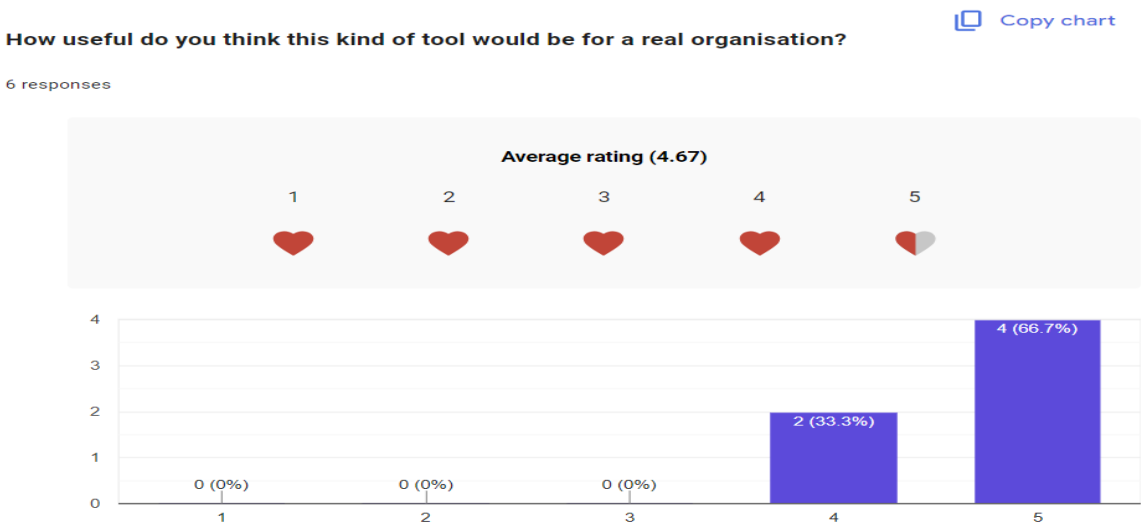

Figure 6.4.1 Usability Trail Q2

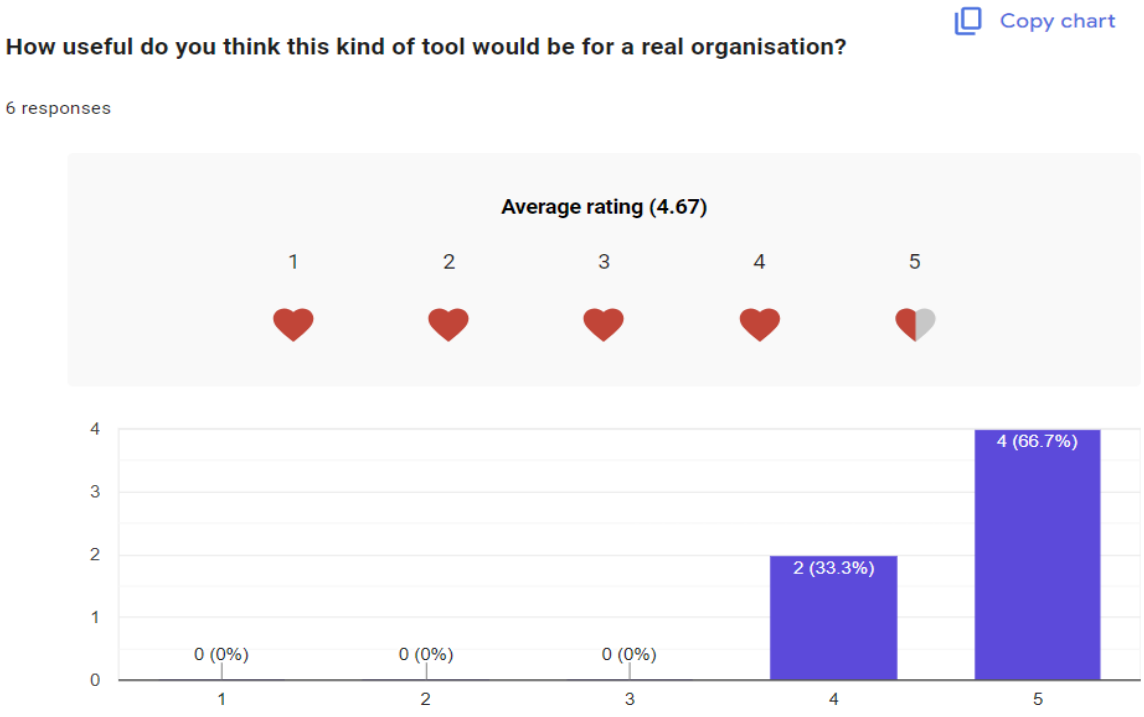

Figure 6.4.2 Usability Trail Q3

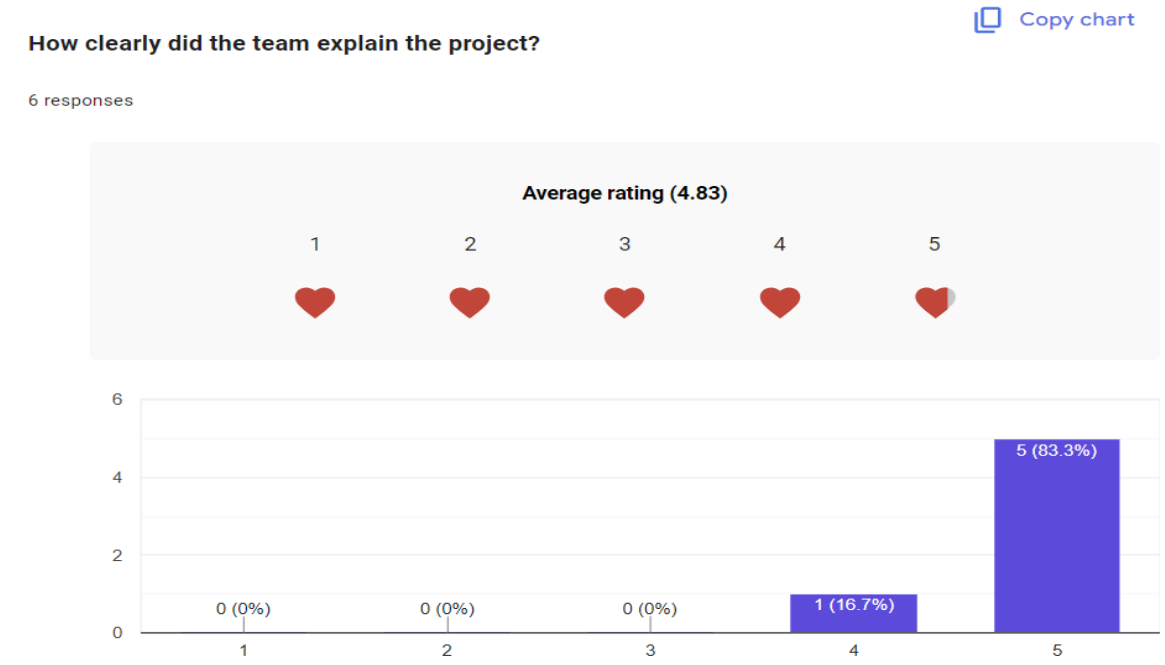

Figure 6.4.3 Usability Trail Q4

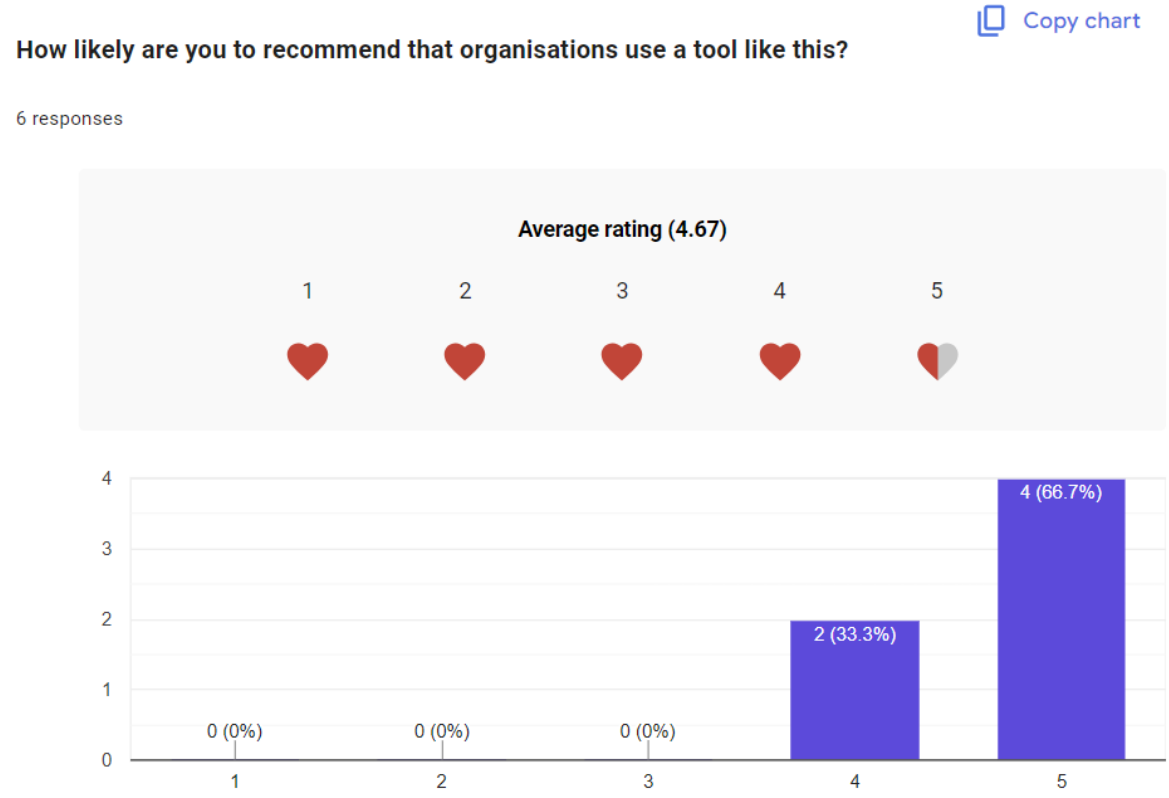

Figure 6.4.5 Usability Trail Q5

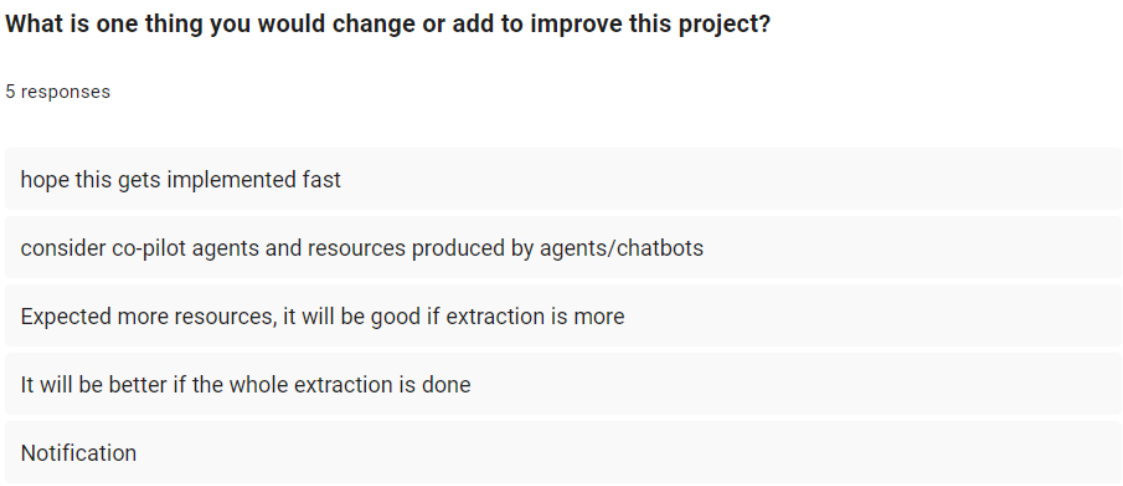

Figure 6.4.6 Usability Trail Q6

6.5 Manual Test Checklist

To verify the tool is working correctly after any code changes or redeployment, run through the steps below. This checklist covers the full extraction cycle from authentication through to output validation. Each step has a clear expected result so you can immediately tell whether something has gone wrong. If any step fails, check the logs/sentinel_TIMESTAMP.log file first - it will usually tell you exactly which module failed and why.

Table 6.5 Manual Verification Steps

Step	Action	Expected Result
1	Run python main.py from handover_backend/	No errors, completes in 2 to 5 minutes
2	Check the /output/ folder	9 files created with today's timestamp in the name
3	Open extraction_history.xlsx	New row added with today's date and record counts
4	Open the audit_logs signin SUMMARY file	Shows sign-in totals: overall, successful, failed, risky
5	Open the file_metadata SUMMARY file	Shows file count, sharing status, sensitivity label coverage
6	Open the sharing_permissions SUMMARY file	Shows permission count, anonymous link count, external users
7	Check dataQualityScore in any output file	Score should be 70 or above for a valid run

8	Open Power BI Desktop and click Refresh	Dashboard tabs update with the latest extraction data

7. Architecture Diagrams

All seven diagrams in this section were created by the team during Sprint 2 and Sprint 3. They show how the different parts of the system connect, from the high-level architecture down to the deployment environment and the class-level code structure.

The System Architecture diagram gives the overall picture how data flows from the MS365 APIs through the extraction engine and into the output files that ShadowSight ingests. The Use Case diagram shows who interacts with the system and what actions they can perform. The Class diagram shows the code structure how the five extractor modules all inherit from BaseExtractor and how the supporting classes like AuthModule, ThrottleHandler, and JSONExporter fit together.

The Sequence diagram walks through a single extraction run step by step, from authentication to export, including how the ThrottleHandler deals with a 429 rate-limit response. The Activity diagram shows the same flow as a process flowchart with decision points and swimlanes. The Component diagram shows the high-level software components and how they depend on each other. Finally, the Deployment diagram shows where each part of the system operates, including the Power Automate scheduler, Azure Entra ID, the Microsoft 365 cloud, the extraction environment, and the ShadowSight platform.

7.1 System Architecture

The system architecture shows the full flow from data sources through to output destinations. At the top are five data sources the tool draws from: Microsoft Graph API, SharePoint REST API, Purview and Compliance API, Admin Endpoints, and PowerShell Modules. Authentication runs through the MSAL Authenticator using OAuth 2.0 Client Credentials and Azure Entra ID non-interactive token exchange. Below that sit three layers: the Extractors (AuditLogExtractor, FileMetadataExtractor, SharingPermissionsExtractor), the Utilities (ThrottleHandler, Logger, JSON Exporter), and the Analytics layer that generates risk flags and governance signals. Output flows to the JSON /output/ folder and then on to ShadowSight at SECMON1 for live monitoring.

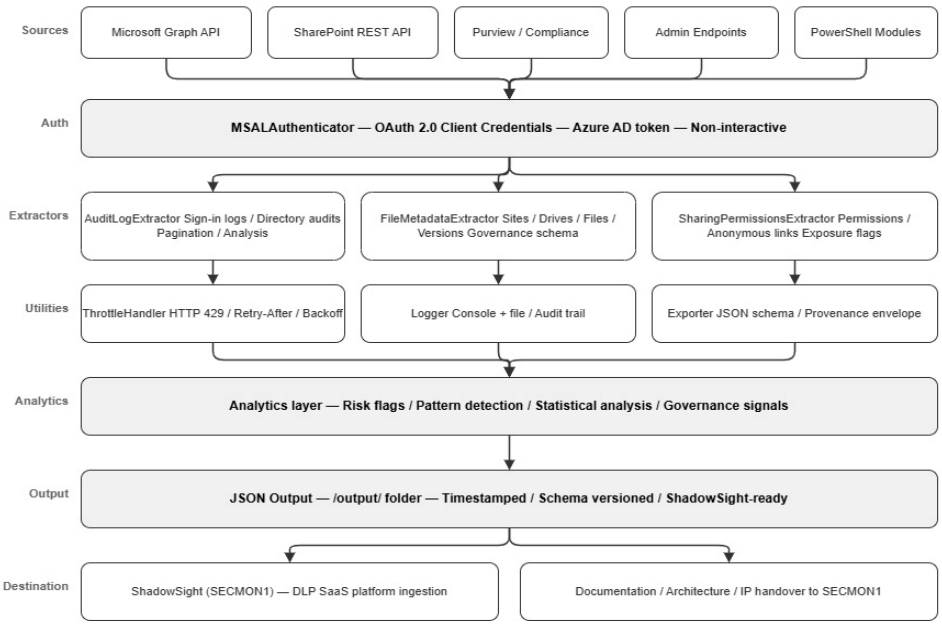

Figure 7.1 System Architecture Diagram

7.2 Use Case Diagram

The use case diagram shows who interacts with the system and what they can do. Two actors are shown: the Scheduler, which triggers automated extraction runs, and the Security Analyst, who reviews the governance findings. The main use cases include running the extraction pipeline, authenticating to Azure Entra ID, extracting audit logs, extracting file metadata, extracting sharing permissions, handling API throttling, logging extraction activity, analysing governance data, flagging governance risks, exporting JSON output, and ingesting the output into ShadowSight.

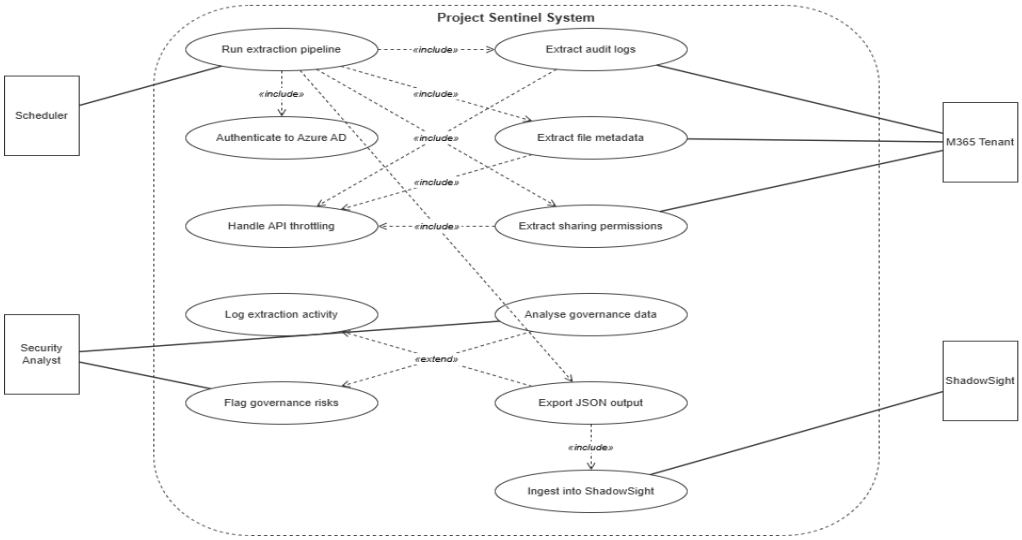

Figure 7.2 Use Case Diagram

7.3 UML Class Diagram

The class diagram shows the object-oriented design of the extraction engine. BaseExtractor sits at the centre as an abstract class with four methods: extract(), analyse(), _paginate(), and normalise(). Four concrete extractor classes inherit from it AuditLogExtractor,FileMetadataExtractor, SharingPermissionsExtractor, and DLPExtractor. Supporting classes are: ExtractionJob (the orchestrator), AuthModule (handles MSAL authentication),APIClient (makes HTTP GET requests with pagination), GovernanceDataset (stores collected records), and ActivityLogger.

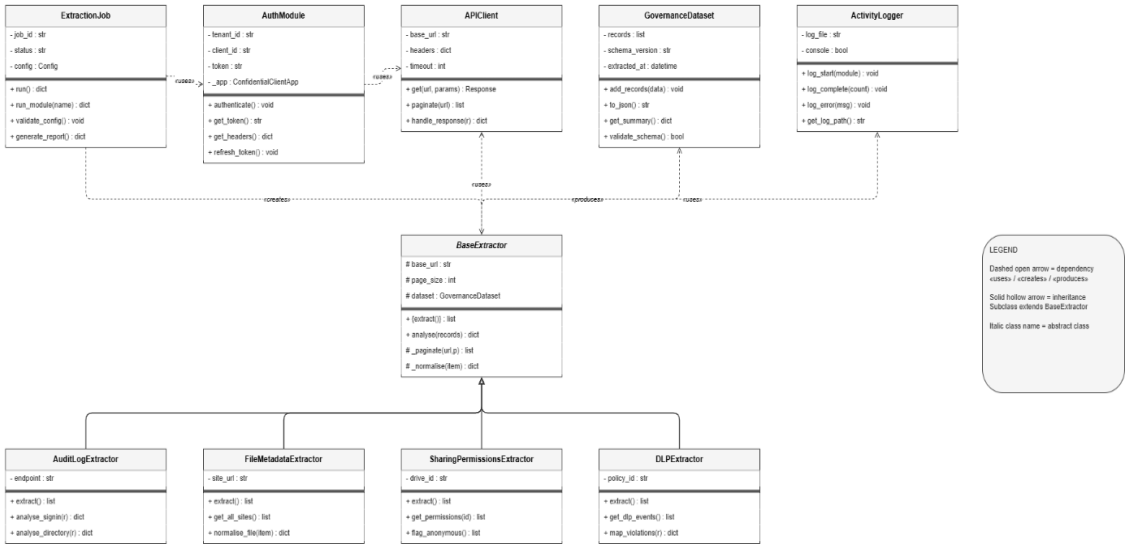

Figure 7.3 UML Class Diagram

7.4 UML Sequence Diagram

The sequence diagram traces the 16 messages that flow between components during a single extraction run. It is split into four phases. Phase 1 (Authentication) shows ExtractionJob calling authenticate() on AuthModule, which POSTs to the token endpoint and gets back an access token valid for 3600 seconds. Phase 2 (Extraction) shows GET /auditLogs/signIns with $top=100, receiving HTTP 429 with a Retry-After header, waiting and retrying, receiving the first page, then looping to follow @odata.nextLink until all pages are collected. Phase 3 shows the governance analysis and risk flags being logged. Phase 4 shows the full dataset being exported to JSON.

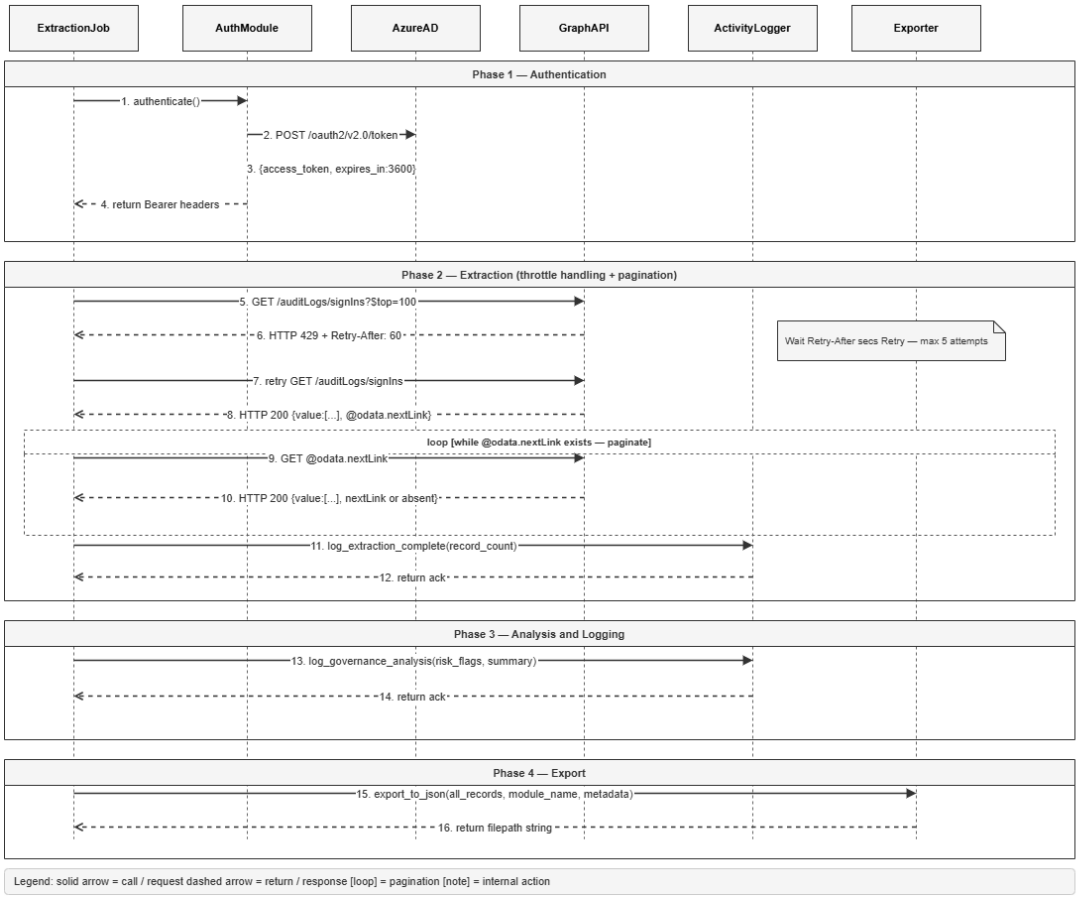

Figure 7.4 UML Sequence Diagram (16 messages)

7.5 Activity Diagram

The activity diagram shows the step-by-step flow of a full extraction run across five swim lanes: Scheduler, ExtractionJob, AuthModule, GraphAPI/ThrottleHandler, and Output Layer. The Scheduler triggers the job. ExtractionJob checks whether the cached token is still valid. If it is, auth headers are set. If not, AuthModule requests a new token from Azure Entra ID. The API call goes to the GraphAPI lane, which checks whether the response is throttled (HTTP 429). If so, it waits Retry-After seconds and retries. Three extractors run in parallel - audit logs, file metadata, sharing permissions then results merge, governance analysis runs, and output is exported to JSON.

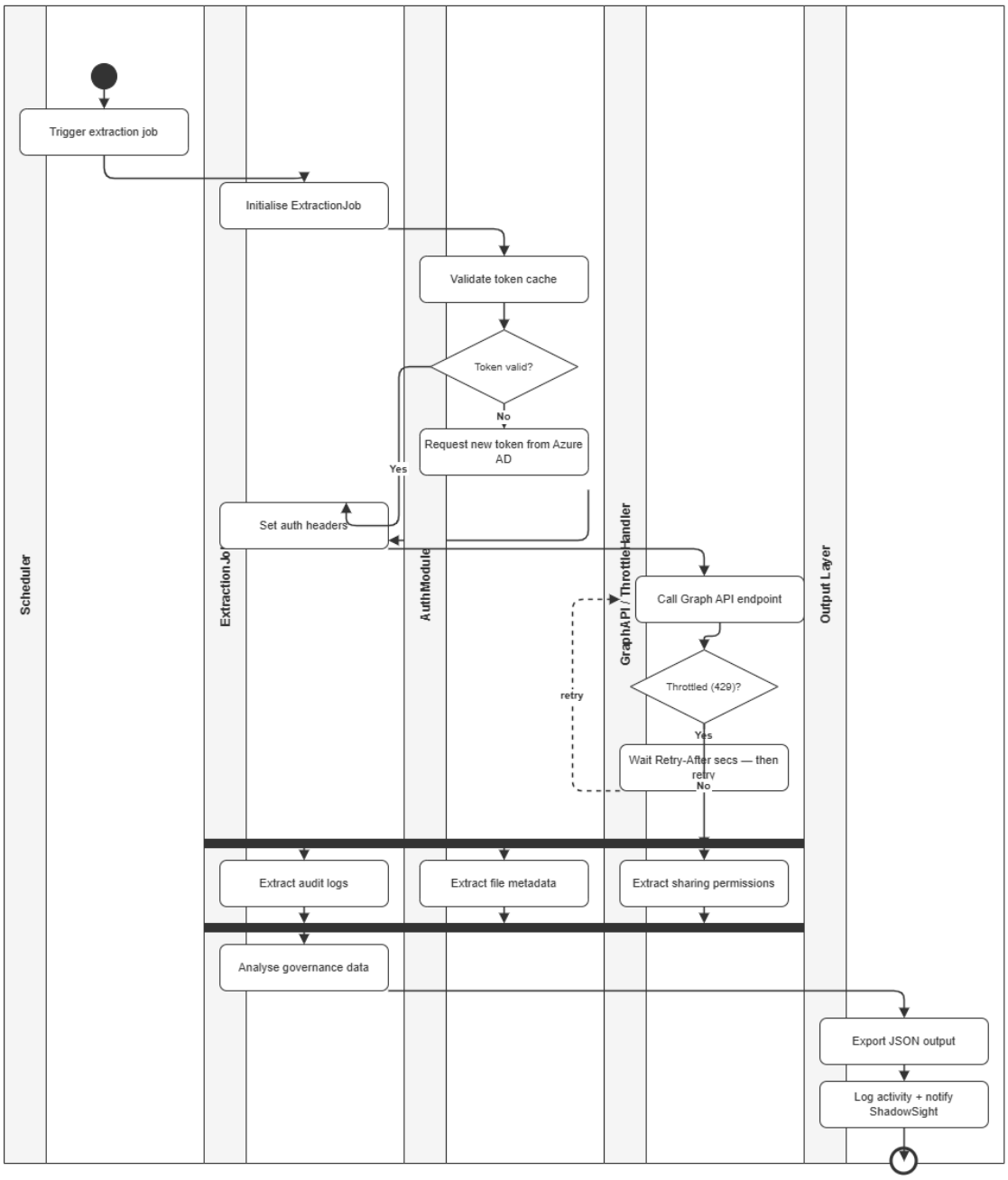

Figure 7.5 Activity Diagram

7.6 Component Diagram

The component diagram groups the system into three internal packages and two external systems. The Extraction Core package contains ExtractionJob, AuthModule, ThrottleHandler, and APIClient. The Analysis and Output package contains GovernanceAnalyser, JSONExporter, and ActivityLogger. The Data Extractors package contains AuditLog Extractor, FileMeta Extractor, Sharing Extractor, and DLP Extractor - all implementing the IExtractor interface. The two external systems are the MS365 Cloud connected via HTTPS/JSON, and the ShadowSight Platform.

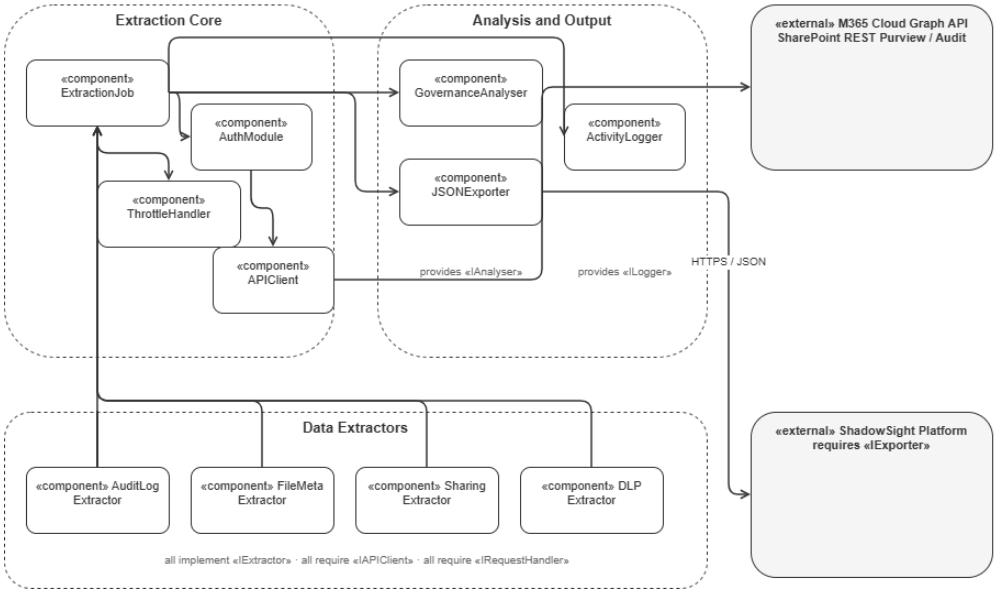

Figure 7.6 Component Diagram

7.7 Deployment Diagram

The deployment diagram shows the environments involved in a full Project Sentinel extraction run. The process begins with the Microsoft Power Automate Scheduled Flow, which automatically triggers the extraction engine each day. The Automation Host contains main.py, auth/msal_auth.py, the extractors folder, config.py, and requirements.txt. It connects to Azure Entra ID using OAuth 2.0 Client Credentials authentication to obtain an access token before calling Microsoft 365 services through Microsoft Graph API.

The Microsoft 365 Cloud provides access to Graph API endpoints, SharePoint and OneDrive data, Unified Audit Logs, and Microsoft Purview services. Extracted data is written to the Output Store as timestamped JSON files and log files. These outputs are then made available for ingestion by the ShadowSight platform for governance monitoring and analysis.

Figure 7.7- Deployment Diagram

7.8 Cyber Threat Model

The table below identifies the main threats to the Project Sentinel system, what could cause them, and what mitigations are in place or recommended. This was prepared as part of the Sprint 5 security analysis by Jaswanthi Palleti (Security Analyst).

The biggest risk in the current setup is the client secret sitting in config.py as plain text. If that file were ever exposed through a misconfigured repository, a leaked ZIP, or a shared screen the entire MS365 tenant would be at risk. The recommended fix is to move the secret into Azure Key Vault before any production deployment. The other risks in the table are lower severity but still worth knowing about, particularly around the output JSON files which contain real governance data and should be treated as sensitive.

Table 7.8 Cyber Threat Model

Threat	Attack Vector	Current Risk	Mitigation
Client secret exposed in source code	Developer commits config.py to a public GitHub repository by accident	High	Never commit config.py. For production, store the secret in Azure Key Vault and use a managed identity to read it.
Overprivileged app permissions	An attacker gains access to the app registration and can access more data than needed	Medium	All permissions are read-only. Sites.FullControl.All was reviewed and removed in. Least-privilege model applied.

Expired client secret causes outage	Monday extraction fails silently because the secret expired	Medium	Set a calendar reminder before the secret expiry date. Monitor extraction_history.xlsx for missed runs.
API response data intercepted in transit	Man-in-the-middle attack intercepts JSON data between the tool and Microsoft APIs	Low	All API calls use HTTPS. Microsoft endpoints enforce TLS. No plain HTTP calls are made anywhere in the codebase.
Output files accessed by unauthorised users	Someone gains access to the Azure storage account and reads extraction output	Medium	Restrict access to the /output/ directory to authorised users only. For production, enable storage encryption and access logging
Microsoft Graph API breaking change	A Graph API endpoint changes its response shape, causing extraction to fail or return wrong data	Low	Monitor the Graph API changelog. The tool pins to v1.0 for stable endpoints. Beta endpoints for sensitivity labels should be reviewed periodically.
Insider threat - data misuse	A team member or future maintainer uses the extracted governance data outside of SECMON1	Low	Output files are scoped to the Azure environment. Access logs are available via Azure Monitor. Data is synthetic in the dev tenant.

8. Data Output Structure

8.1 Schema Overview

The output schema is at version 2.0.0, designed during Sprint 2 by Lahari Borra (Documentation Specialist) with API attribute mapping contributed by Bharanee Raghav Murru (Dashboard Developer). The schema defines 95 fields across 9 categories. Every extraction run produces output files that follow this schema exactly.

Table 8.1 Output Schema Categories

#	Category Name	Description	API Source
1	extractionMetadata	Provenance envelope when, from where, and how the data was pulled	Generated by extraction tool
2	tenantContext	Which tenant and SharePoint site the data came from	GET /sites/{siteId} - Graph API
3	fileGovernance	Core file metadata: name, size, owner, dates, version history	GET /drives/{id}/items - Graph API
4	classificationProtection	Sensitivity labels, DLP matches, encryption, Rights Management	Graph beta + Purview Compliance API
5	retentionRecords	Retention labels, legal holds, and records management status	Graph beta + eDiscovery API (E5 required)
6	sharingAccess	Permissions, sharing links, external users, and exposure level	GET /items/{id}/permissions Graph API
7	auditActivity	Array of audit log events per file who did what and when	GET /security/auditLog Graph API
8	ownershipLifecycle	Inactivity days, orphaned file flag, lifecycle stage	Derived from file metadata and audit logs
9	qualityIndicators	Data completeness score, validation result, estimated fields flag	Generated by extraction tool

8.2 Provenance Envelope Structure

Every JSON output file begins with an extractionMetadata block the provenance envelope. ShadowSight reads this block first to validate the file before processing the records inside. A typical provenance envelope looks like this:

```
{
  "extractionMetadata": {
    "extractionId":              "ext-20260518073754001",
    "extractionTimestamp":       "2026-05-18T07:37:54Z",
    "tenantId":                          "a3b8c9d0-xxxx-xxxx-xxxx-
xxxxxxxxxxxx",
    "tenantName":                "Sentinals.onmicrosoft.com",
    "extractionToolVersion":     "1.0.0",
    "schemaVersion":             "2.0.0",
    "extractionScope":           "full_tenant",
    "totalRecordsExtracted":     354,
    "extractionDurationSeconds": 187,
    "apiThrottlingEncountered":  true,
    "dataQualityScore":          78,
    "authenticationType":        "service_principal_secret"
  },
  "records": [ ... ]
}
```

8.3 Output Schema Categories

The table below covers the most important fields from each category. The full 95-field specification is in Ouput_Schema_Specification.json.

Table 8.3 Key Output Fields by Category

Category	Key Fields	Required
fileGovernance	fileId, fileName, filePath, owner (userId, displayName), createdDate,lastModifiedDate,fileSizeBytes, itemWebUrl	Yes
fileGovernance	versionCount, checkoutStatus	No
classificationProtection	sensitivityLabel (labelId, labelName, appliedDate, applicationMethod),encryptionStatus, rightsManagementApplied	Partial
classificationProtection	dlpMatches (policyId, ruleNames, severity, actionTaken, sensitiveInfoType)	No - E5 required
sharingAccess	directPermissions (permissionId, grantee, permissionLevel, isExternal, isInherited)	Yes

sharingAccess	externalSharingLinks (linkType, isAnonymousLink, accessScope, expirationDate)	Yes
sharingAccess	guestAccessCount, externalUserCount, exposureLevel	Yes
auditActivity	activityId, userId, userPrincipalName, actionType, timestamp, resultStatus, ipAddress	Yes
ownershipLifecycle	inactivityDays, orphanedFile, lifecycleStage (active / inactive_30d / stale_1yr / orphaned)	No
qualityIndicators	dataCompleteness (0–100), isValidated, hasEstimatedData, lastValidated	Yes
retentionRecords	retentionLabel, recordDeclarationStatus, legalHoldApplied, holdType, caseId	No - E5 for eDiscovery

8.4 Exposure Level Logic

The exposureLevel field in sharingAccess is calculated automatically by the extraction tool based on two things how many external users have access to a file and whether any anonymous sharing links exist. It is not pulled from the MS365 API directly. The SharingPermissionsExtractor calculates it after collecting the raw permission data.

The field has three possible values: internal, limited_external, and wide_external. These give SECMON1 a quick way to filter and prioritise files without having to count external users manually. A file with an exposureLevel of wide_external should always be reviewed first as it means the content is accessible to a large number of outside users or is openly shared via an anonymous link.

The table below shows exactly how each level is calculated:

Table 8.4 Exposure Level Calculation Rules

Exposure Level	Condition
internal	No external users and no anonymous sharing links
limited_external	1 to 5 external users with any level of access
wide_external	6 or more external users, OR at least one anonymous link exists

In both live extraction runs (Sprint 4 and Sprint 5), all files showed an exposureLevel of internal. The developer tenant had no external sharing configured and zero anonymous links.

9. Project History

9.1 Development Methodology

The team followed the Scrum agile methodology across six sprints. Each sprint had a rotating Scrum Master who was responsible for leading the weekly planning session, tracking Jira tasks, and presenting the sprint outcomes. The diagram below shows the full Scrum cycle the team used and how each sprint fits into the overall project timeline.

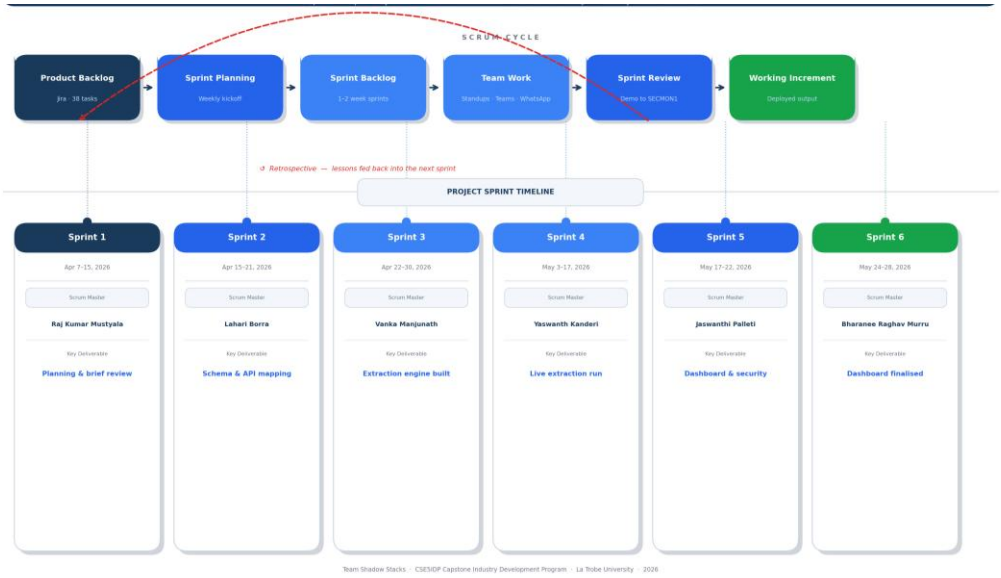

Figure 9.1 Scrum Methodology and Sprint Timeline

9.2 Implementation Methodology

The diagram below shows the full technical implementation pipeline for Project Sentinel how a single extraction run flows from the Power Automate trigger all the way through to ShadowSight ingestion. Each stage is handled by a dedicated component in the codebase.

The pipeline starts with the Power Automate scheduled flow firing daily at 8am. This triggers the extraction engine which first authenticates with Azure Entra ID using MSAL OAuth 2.0 Client Credentials flow to get a Bearer token. The five extraction modules then run in sequence each one calls the relevant MS365 API endpoints, handles pagination using the @odata.nextLink field, and passes any HTTP 429 throttle responses to the ThrottleHandler which waits and retries up to 5 times. Once extraction is complete, the Governance Analyser runs risk scoring and exposure level calculations on the collected data.

The JSON Exporter then writes nine timestamped output files to the /output/ directory and updates extraction_history.xlsx. The Activity Logger records every step throughout the run. Finally, ShadowSight ingests the output files via HTTPS or SFTP for governance monitoring and alerting.

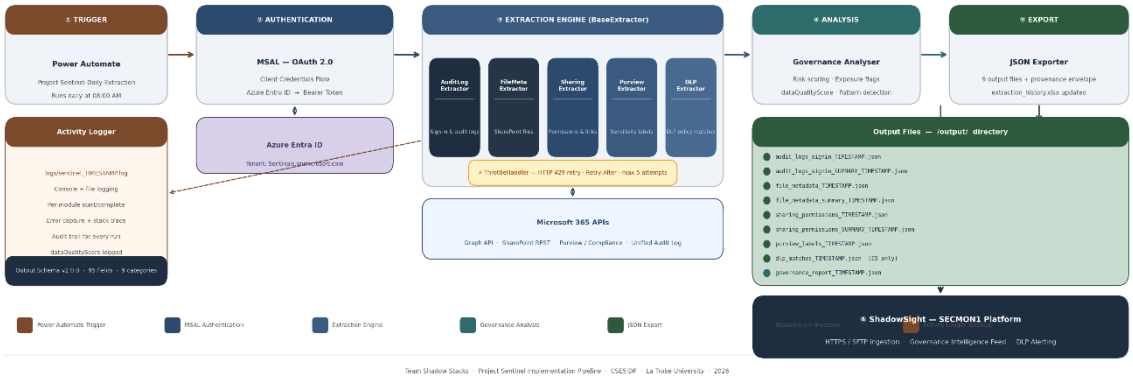

Figure 9.2 Project Sentinel Implementation Pipeline

9.3 Team and Roles

Team Shadow Stacks worked on Project Sentinel across six sprints as part of La Trobe University's CSE5IDP Capstone Industry Development Program. The industry partner was Christopher McNaughton from SECMON1.

Table 9.3 Team Shadow Stacks Members

Name	Student ID	Role	Key Contributions
Raj Kumar Mustyala	22234543	Team Lead / Project Manager	Sprint 1 Scrum Master. Security attribute mapping, sensitivity label config, sprint coordination, app registration research,. Sprint 6: final review and project alignment.
Yaswanth Kanderi	22519671	Backend Developer / Sprint 4 Scrum Master	MS365 developer tenant setup, SharePoint config, Microsoft Purview setup, RBAC and role assignment, audit log validation. Sprint 6: data integrity validation and dashboard metrics verification. Sentinel-Teams-DLP-Policy configuration in Microsoft Purview.
Vanka Manjunath	22197477	API Developer / Sprint 3 Scrum Master	Built MSAL auth module, 5 extraction modules, throttle handler, logger, JSON exporter, main orchestrator. Azure Function

			App deployed. Sprint 6: API data structuring and refinement for dashboard.
Bharanee Raghav Murru	22003177	Dashboard Developer / Sprint 6 Scrum Master	API attribute mapping, risk scoring framework, Sprint 3 data analysis. Sprint 6: built and finalised 5-page Power BI governance dashboard, fixed 3 data accuracy bugs using DAX DISTINCTCOUNT measures.
Jaswanthi Palleti	22115229	Security Analyst / Sprint 5 Scrum Master	Governance data identification, API permission model, governance attribute tables, Sprint 5 security analysis and 5 recommendations. Sprint 6: defined governance and security insights for dashboard.
Lahari Borra	22134698	Documentation Specialist / Sprint 2 Scrum Master	Output schema design, system maintenance document, user documentation, usability trial, poster, data validation rules. Sprint 6: final documentation and project outcomes.

9.4 Communication Channels

The team used two main communication channels throughout the project. Microsoft Teams was the primary platform for all formal communication sprint planning meetings, weekly check-ins, and any contact with the industry partner Christopher McNaughton. File sharing, meeting notes, and sprint deliverable handovers were all done through Teams.

WhatsApp was used for day-to-day coordination between team members. Quick questions, reminders about deadlines, and status updates during busy sprint periods all went through the WhatsApp group. This kept the Teams workspace cleaner and made it easier to separate formal project work from quick team conversations.

Table 9.4 Team Communication Channels

Channel	Platform	Used For	Participants
Microsoft Teams	Microsoft Teams (La Trobe)	Sprint planning, weekly check-ins, industry partner meetings, file sharing, formal sprint handovers and review	All 6 team members + Christopher McNaughton (SECMON1)
WhatsApp Group	WhatsApp	Day-to-day coordination, quick status updates, deadline reminders, blockers	All 6 team members

		and help requests between team members	

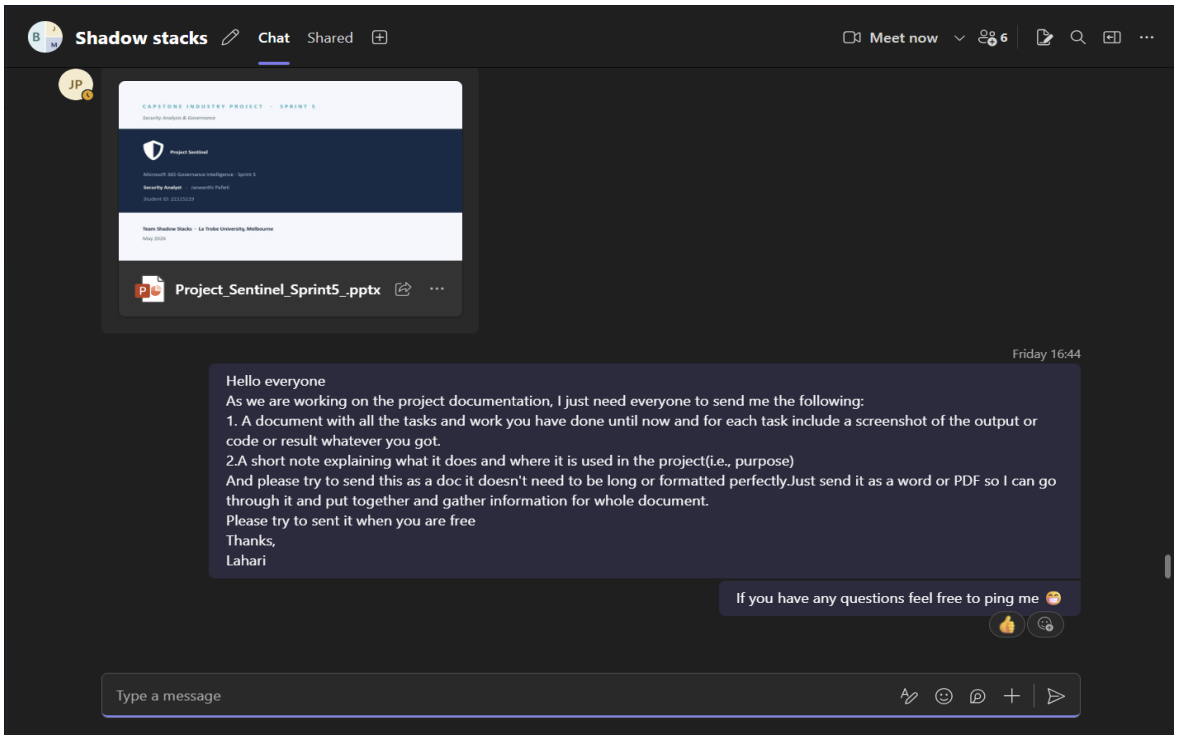

Figure 9.4.1 Microsoft Teams Communication Channel

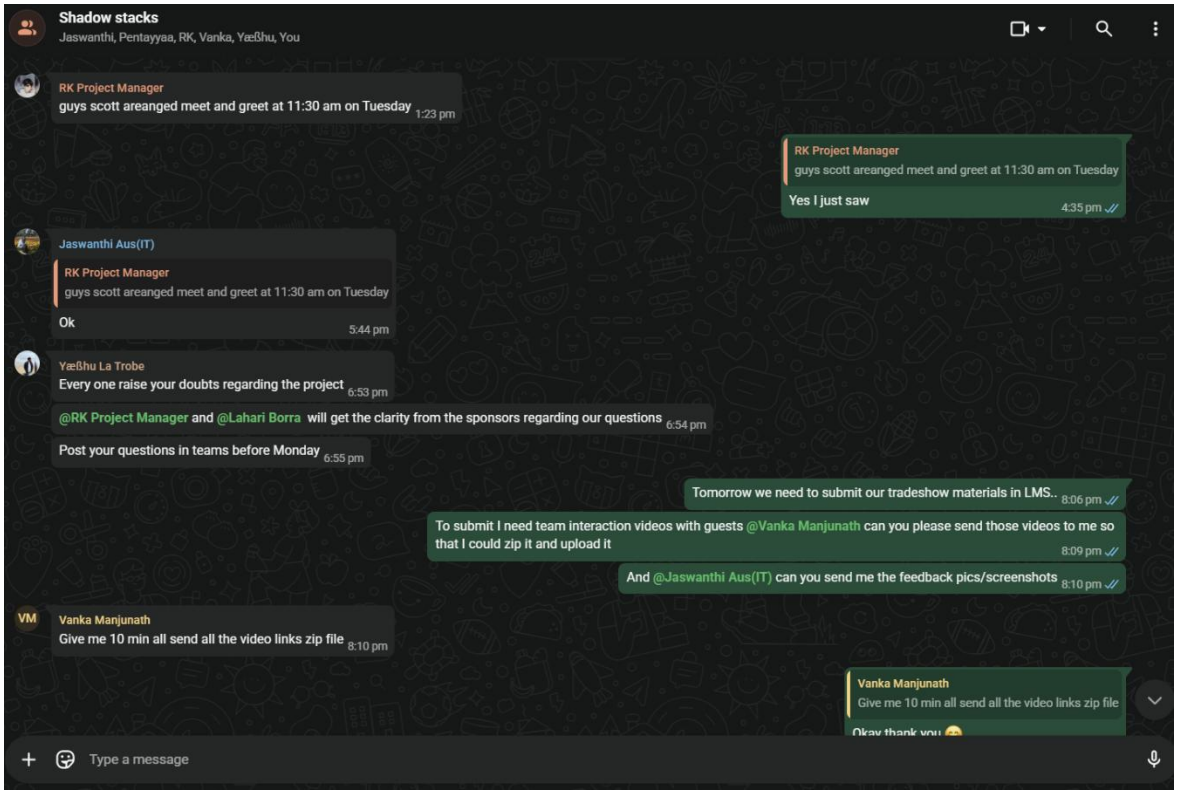

Figure 9.4.1 WhatsApp Group Chat

9.5 Sprint History

Table 9.5 Sprint Summary

Sprint	Dates	Scrum Master	Key Deliverables
Sprint 1	March - April 12, 2026	Raj Kumar Mustyala	Project brief reviewed, team roles assigned, Jira board set up, initial API research, communication channels established, tradeshow participation
Sprint 2	April 13 - 20, 2026	Lahari Borra	Output Schema v2.0.0 (95 fields, 9 categories), Graph API permissions list, architecture diagrams (6 UML views), tender document
Sprint 3	April 22 - 30, 2026	Vanka Manjunath	Python extraction engine (5 modules + BaseExtractor), MSAL auth module, throttle handler, Azure Function App deployed, usability trial, maintenance document v1, risk scoring framework
Sprint 4	May 3 - 17, 2026	Yaswanth Kanderi	Power Automate scheduled flow set up (Project-Sentinel-Daily-Extraction), live extraction run: 183 sign-ins, 18 files, 72 permissions, throttle handling confirmed
Sprint 5	May 17 - 22, 2026	Jaswanthi Palleti	Second live extraction: 354 sign-ins, 26 files, 105 permissions; Power BI dashboard built (5 tabs); governance risk analysis; 5 security recommendations made; Sentinel-Teams-DLP-Policy configured in Microsoft Purview (simulation mode).
Sprint 6	May 24 - 28, 2026	Bharanee Raghav Murru	Power BI dashboard finalised (5 pages, 34 story points, 6 tasks DONE). 3 data bugs fixed using DAX measures. DLP policy simulation confirmed - Australia Tax File Number detected and alert triggered (31 May 2026). Final documentation completed. Final review and project alignment done. Project Sentinel ready for SECMON1 handover.

9.6 Jira Task Summary

All project tasks were tracked in Jira on the board CSE5IDP-ShadowStacks_Fri_11-1. The team completed 32 tasks across the six sprints - 32 from Sprints 1 to 5 and 6 in Sprint 6. All 32 tasks are marked DONE as we don't have the Sprint 1 task not recorded in team Jira Page. Tasks were created at sprint start, assigned to individual team members, and reviewed at the end of each sprint during the Scrum Master presentation.

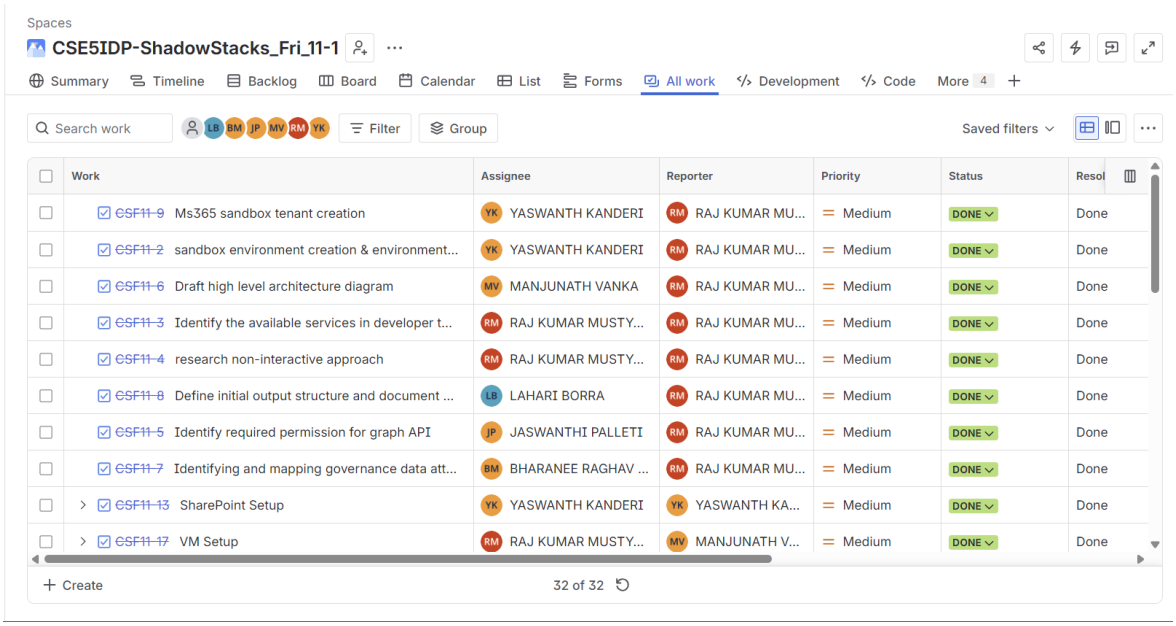

Figure 9.6.1 Jira Board: All 32 Tasks Complete

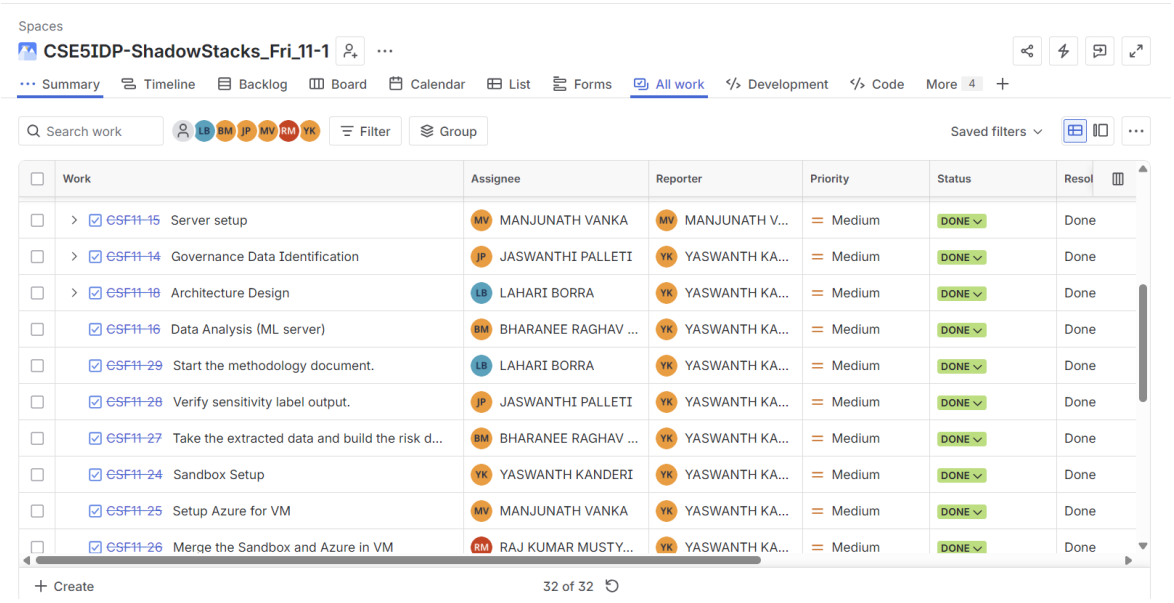

Figure 9.6.2 Jira Board: All 32 Tasks Complete

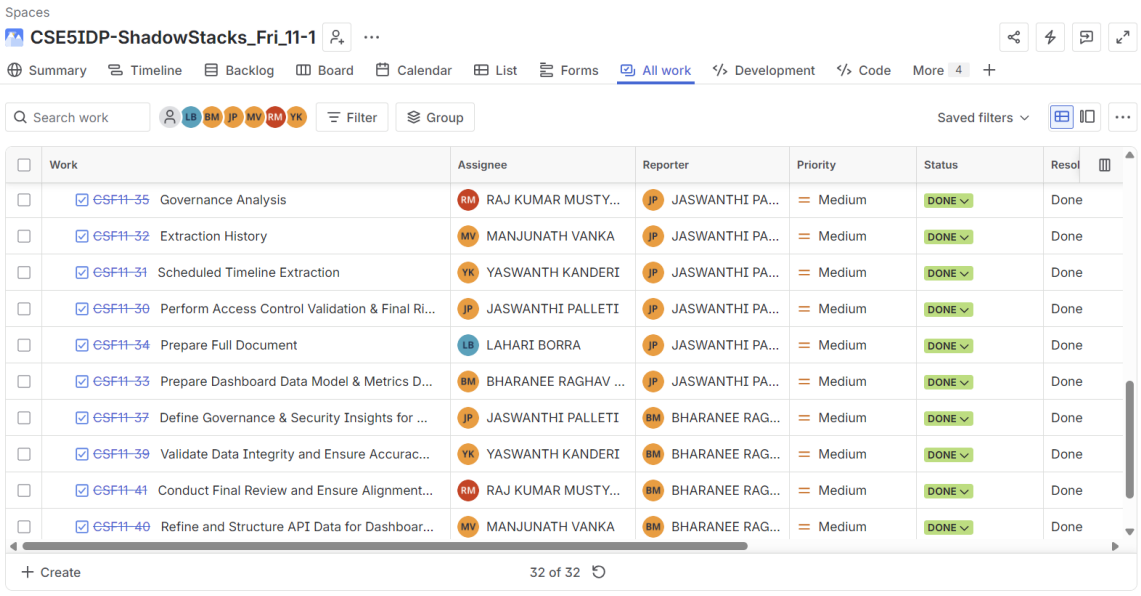

Figure 9.6.3 Jira Board: All 32 Tasks Complete

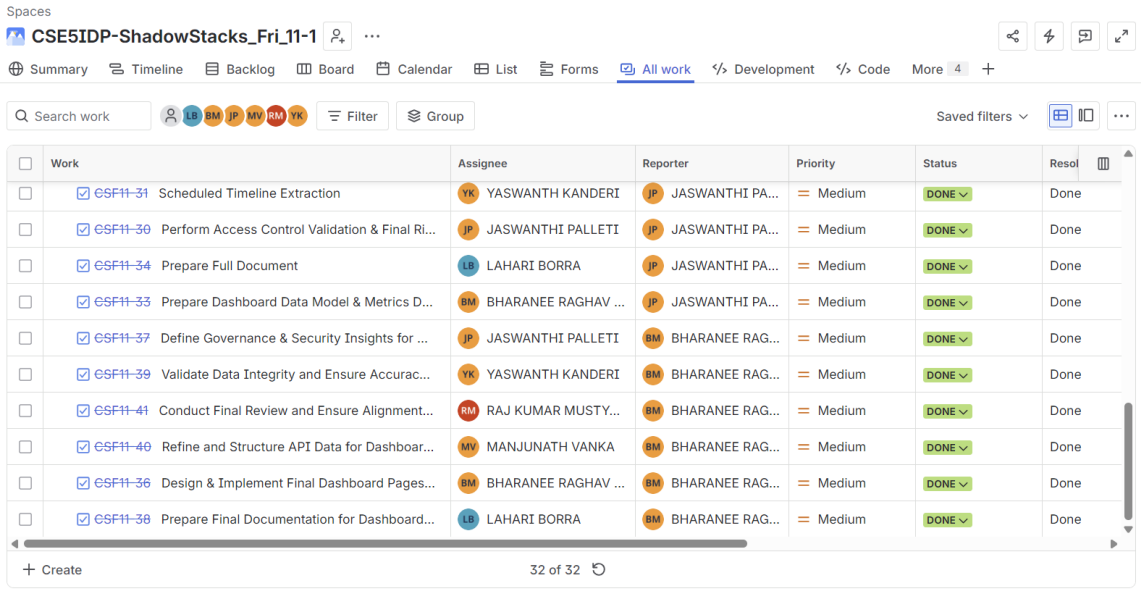

Figure 9.6.4 Jira Board: All 32 Tasks Complete

9.7 Power BI Dashboard

The Power BI dashboard was designed and built by Bharanee Raghav Murru (Dashboard Developer) in Sprint 6 using data from the Sprint 5 live extraction. The dashboard file is Project_Sentinel_Dashboard.pbix. It connects directly to the JSON output files produced by the extraction engine and has five pages, each covering a different governance area. The build was completed in Sprint 6 with 34 story points delivered across 6 tasks.

During Sprint 6, three data accuracy issues were found and fixed. The Total Files card was showing 408 (the row count) instead of 26 distinct files fixed by applying a DISTINCTCOUNT(file_name) DAX measure. The Total Users card was connected to the

wrong column fixed with a DISTINCTCOUNT on the User Name column, now correctly showing 5 users. Two test accounts were appearing in the user charts fixed by applying a page-level filter to exclude them.

Tab 1 - Audit Monitoring

This tab shows total logins (1.247K), a bar chart of most active users by login count (Yaswanth Kanderi, RK, Vanka Manjunath, Jaswanthi P, Lahari B), a login activity over time line chart, and a File Sharing Exposure Distribution pie chart showing Restricted 40 files (9.8%), Editable 101 files (24.75%), and Internal 267 files (65.44%).

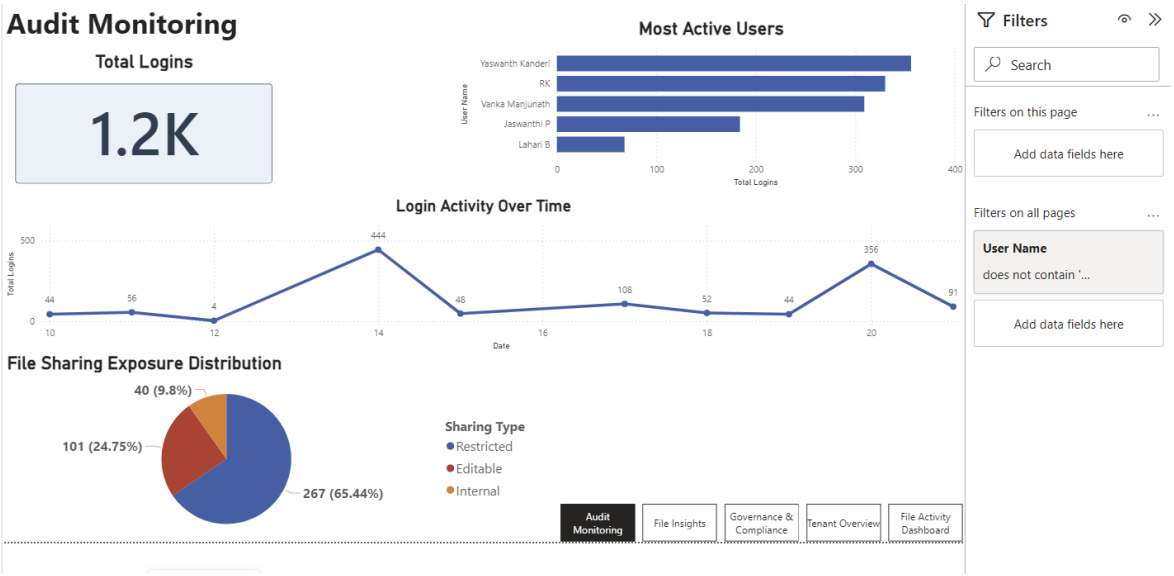

Figure 9.7.1 Power BI Dashboard: Audit Monitoring Tab

Tab 2 File Insights

This tab shows total files (26), a donut chart of files with and without passwords (90.2% have no password, 9.8% have a password), a Shared Files Overview bar chart listing file names by access frequency, and a Files by Link Type bar chart.

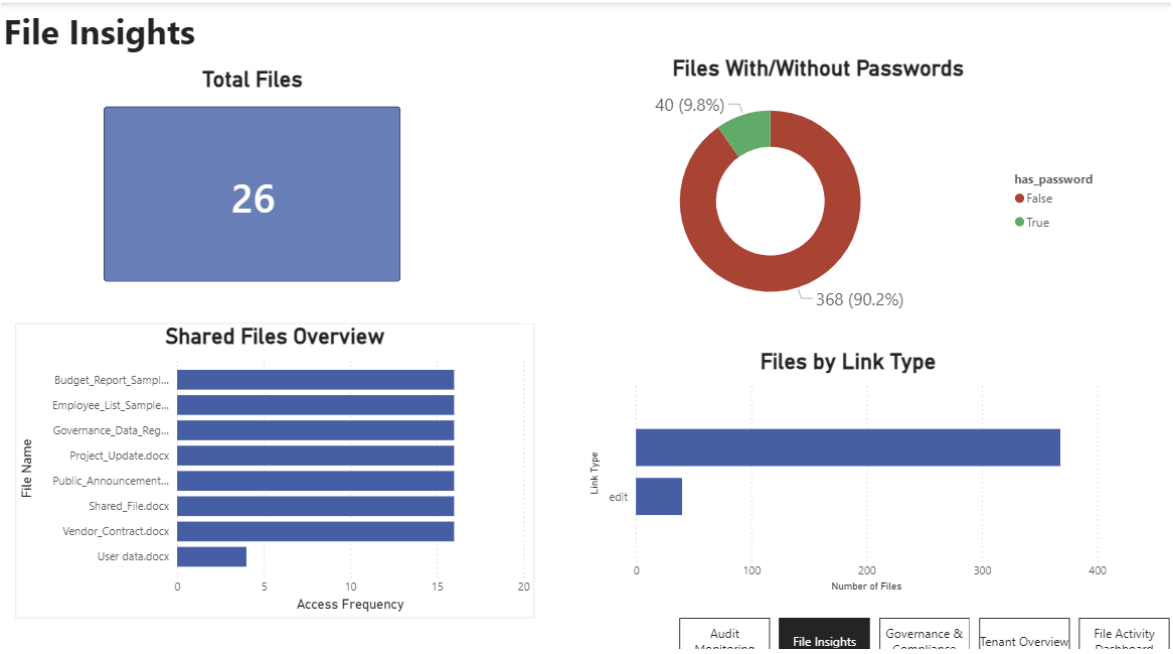

Figure 9.7.2 Power BI Dashboard: File Insights Tab

Tab 3 Governance and Compliance

This tab shows total shared records (408), Risk Level MEDIUM, and Compliance Visibility PARTIAL. Risk factors listed include: high number of editable files, frequent file access activity, and lack of classification controls. Compliance status notes that audit logging is enabled, the Sentinel-Teams-DLP-Policy is active in simulation mode (successfully detected an Australia Tax File Number on 31 May 2026), and no sensitivity labels have been applied to any files.

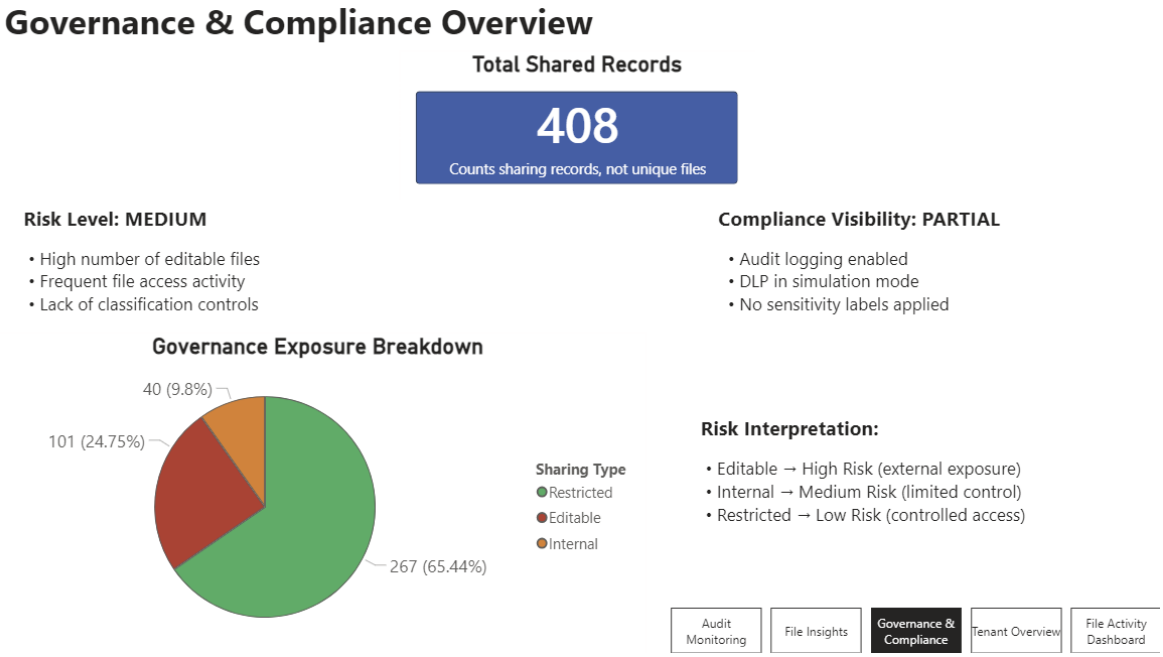

Figure 9.7.3 Power BI Dashboard: Governance and Compliance Tab

Tab 4 Tenant Overview

This is the headline summary tab. It shows four KPI cards: Total Files (26), Total Users (5), High-Risk Files (101), and Total Audit Events (1.2K). The tenant summary notes: 26 files shared across 5 users, 101 files classified as high-risk due to editable sharing permissions, audit logging active with approximately 1.2K recorded events, overall risk assessed as Medium.

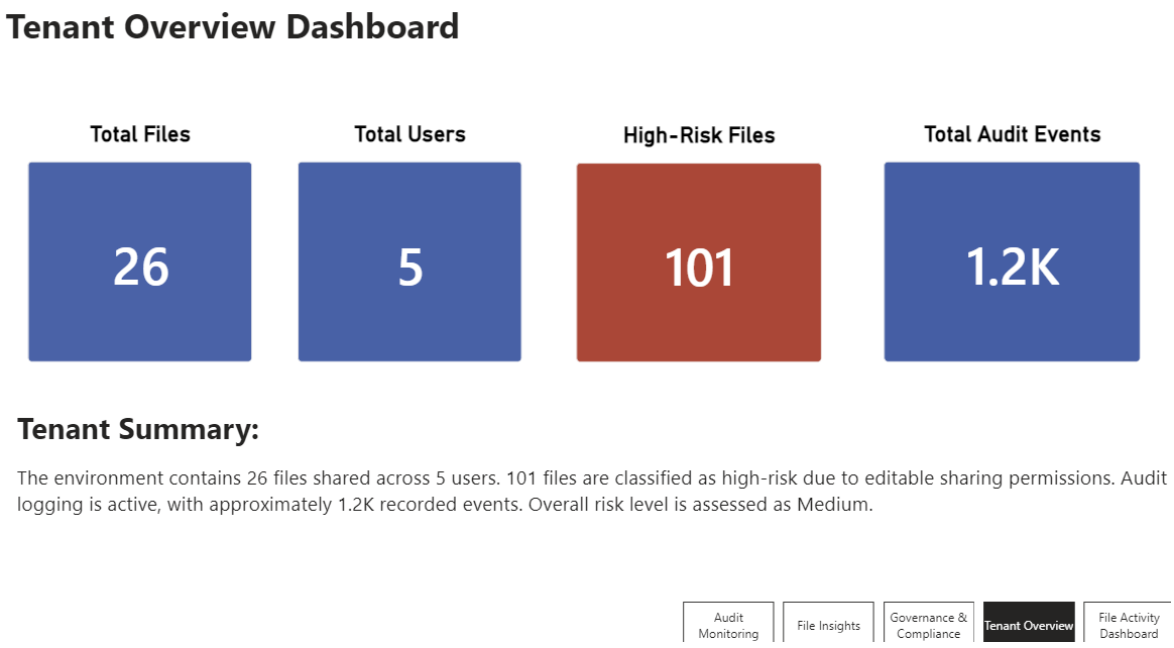

Figure 9.7.4 Power BI Dashboard: Tenant Overview Tab

Tab 5 File Activity Dashboard

This tab shows a File Activity Trend line chart over time, a User Activity Distribution bar chart (Yaswanth Kanderi 356 logins, RK 330, Vanka Manjunath 309, Jaswanthi P 184, Lahari B 68), a File Access Distribution bar chart showing access counts per file name, and a Most

Active User card highlighting Yaswanth Kanderi.

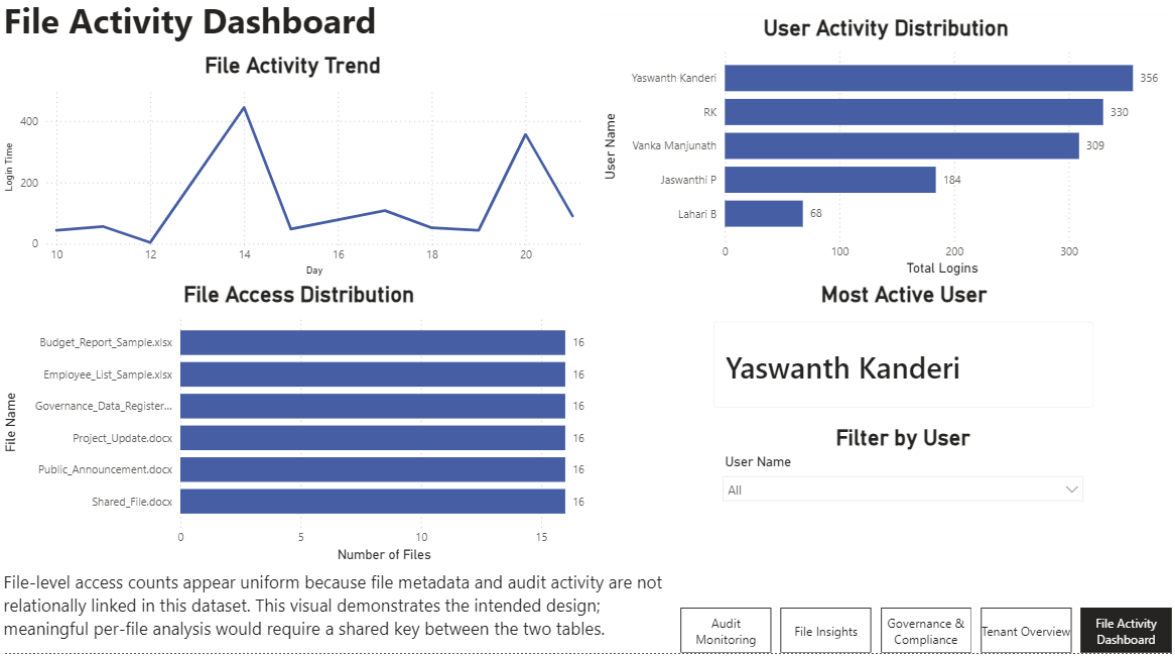

Figure 9.7.5 Power BI Dashboard: File Activity Dashboard Tab

9.8 Lessons Learned

API permission research should start in Sprint 1. Getting all the Microsoft Graph API permissions granted took longer than expected the tenant admin needed to approve each one. Starting this work earlier would have unlocked the live extraction steps sooner.

E3 versus E5 licensing affects what data you can actually pull. We designed the schema with full DLP and eDiscovery fields assuming an E5 tenant, but the developer tenant ended up being E5. Setting license expectations before finalising the schema would have helped us make clearer promises about what the tool would deliver from day one.

Writing documentation continuously is much easier than writing it all at the end. Each sprint produced an update to the maintenance document and the user documentation, which meant the Sprint 6 final write-up was a refinement rather than a scramble.

Thursday weekly check-ins kept the team aligned. When blockers came up such as the pagination issue during Sprint 4 the team knew about it within the same week and could reassign support.

The Sites.FullControl.All permission flag in Sprint 2 was a good example of the team doing proper security thinking early. Catching that before the code was written, not after, meant we never built the extraction engine with over-privileged access.

10. Appendices

10.1 Glossary of Terms

Table 10.1 Glossary

Term	Definition
Azure Entra ID	Microsoft's cloud identity service, formerly called Azure Active Directory. Manages app registrations, user identities, authentication, and API permissions.
MSAL	Microsoft Authentication Library - the Python package used to request OAuth 2.0 access tokens from Azure Entra ID.
Client Credentials	An OAuth 2.0 authentication flow where an application authenticates using its own client ID and client secret, without any user logging in.
Bearer token	The access token returned by Azure Entra ID. Attached to every API request in the HTTP Authorization header as Bearer {token}.
Microsoft Graph API	Microsoft's primary REST API for accessing Microsoft 365 data - files, users, audit logs, permissions, sensitivity labels, and more.
Provenance envelope	The extractionMetadata block at the top of every output JSON file. Records when, where, and how the data was extracted for chain-of-custody and validation.
BaseExtractor	The abstract Python class all five extractor modules inherit from. Provides shared logic for API calls, pagination, throttle handling, and JSON export.
ThrottleHandler	A component built into BaseExtractor that handles HTTP 429 (Too Many Requests) responses by reading the Retry-After header, waiting, and retrying up to 5 times.
@odata.nextLink	A URL field in Microsoft Graph API responses pointing to the next page of results. The extraction engine follows this field until it is absent.
Sensitivity label	A Microsoft Purview classification tag applied to a file (e.g., Confidential, Internal Only). Labels control what protections are applied to the content.
DLP (Data Loss Prevention)	Microsoft Purview policies that detect sensitive information types (e.g., credit card numbers, Australian Tax File Numbers) in files and take configured actions. In this

	project, the Sentinel-Teams-DLP-Policy monitors Exchange email, SharePoint, OneDrive, Teams, and on-premises repositories in simulation mode.
Power Automate	Microsoft's workflow automation platform. Used in this project to schedule the daily extraction run via the Project-Sentinel-Daily-Extraction flow.
ShadowSight	The SECMON1 platform that ingests the JSON output from Project Sentinel for governance monitoring, alerting, and investigation.
SECMON1	The industry partner for this project. Christopher McNaughton from SECMON1 provided the project brief and requirements.
Purview	Microsoft Purview - Microsoft's compliance and information governance platform, providing sensitivity labels, retention labels, and DLP policy management.
Extraction run	One complete execution of the pipeline: authenticate, extract all five modules, analyse, export. Runs automatically every day at 08:00 AM via the Power Automate scheduled flow.
dataQualityScore	A 0 to 100 integer calculated after each extraction run. 100 means every required field was populated and passed validation.
exposureLevel	A calculated field (internal / limited_external / wide_external) showing how broadly a file is accessible.
CVE	Common Vulnerabilities and Exposures - a public list of known security vulnerabilities in software packages. Run pip-audit to check your installed packages.
least-privilege	A security principle meaning each component only gets the minimum permissions it needs to do its job no more.

10.2 Sprint Deliverables Index

Table 10.2 Key Deliverables by Sprint and Contributor

Sprint	File / Deliverable	Contributor	Description
Sprint 1	Project_Sentinel_sprint1.pptx	Raj Kumar	Initial project planning, team role allocation, brief review, Jira setup discussion, and early dashboard planning for Microsoft 365 governance.
Sprint 1	Project_Sentinel_Tender.docx	Whole team	Tender response document outlining the proposed solution and delivery plan
Sprint 2	Data_Validation_Rules.txt	Lahari Borra	Validation rules and data quality scoring methodology for all JSON outputs
Sprint 2	Sprint_2_Outcomes_Summary_FINAL.docx	Lahari Borra	Sprint 2 outcomes summary covering schema design, permission mapping, and architecture
Sprint 2	Graph*API*_Permissions.docx	Jaswanthi Palleti	Microsoft Graph API permission mapping and security classification for governance data extraction.
Sprint 3	handover_backend/ (Files_Backend.zip)	Vanka Manjunath	Complete Python extraction engine 5 modules, BaseExtractor, MSAL auth, ThrottleHandler, Azure Function App
Sprint 3	Project Sentinel Document.docx	Vanka Manjunath	Source file with all 7 UML and architecture diagrams
Sprint 3	Governance_Data_Identification.pdf	Jaswanthi Palleti	SharePoint metadata fields and governance attribute mapping tables
Sprint 3	Sentinel_Usability_Trial_Final.pptx	Lahari Borra	Usability trial presentation tasks, findings, and documentation improvements

Sprint 3	Sprint3_deliverables.pdf	Vanka Manjunath	Sprint 3 combined deliverables system architecture and component definitions
Sprint 4	sprint-4.pptx	Yaswanth Kanderi	Sprint 4 presentation live extraction results (183 sign-ins, 18 files, 72 permissions)
Sprint 4	Sentinel_User_Document_Final.docx	Lahari Borra	Full user documentation setup, extraction walkthrough, output guide, and troubleshooting
Sprint 5	Project_Sentinel_Sprint5_.pptx	Jaswanthi Palleti	Sprint 5 presentation security analysis, permission risks, and 5 recommendations
Sprint 6	Project_Sentinel_Dashboard.pbix	Bharanee Raghav Murru	Final Power BI governance dashboard - 5 pages, 34 story points, 3 DAX bugs fixed, accurate KPIs (26 files, 5 users, 101 high-risk, 1.247K events)
Sprint 6	Sentinal_Maintainance_Final.docx	Lahari Borra	Final System Maintenance Documents all sections complete, methodology and architecture diagrams embedded.
Sprint 6	Governance Security Analysis Outputs	Jaswanthi Palleti	Provided governance, compliance, permissions, sensitivity label, audit traceability, and risk analysis outputs used as input for final dashboard metrics and visualisations.
Sprint 6	Project_Sentinel_sprint6.pptx	Bharanee Raghav Murru	Final Sprint 6 presentation covering dashboard progress, story points, Power BI screenshots, documentation completion, final review, and overall project completion summary.
All Sprints	extraction_history.xlsx	Vanka Manjunath	Cumulative extraction run log with timestamps, record counts, and data quality scores

All Sprints	GitHub Repository github.com/YaswanthKanderi/Project-Sentinel	Yaswanth Kanderi	Full source code repository with complete version history of the extraction engine

10.3 References

Microsoft Corporation. (2024). Microsoft Graph API documentation. https://learn.microsoft.com/en-us/graph/api/overview

Microsoft Corporation. (2024). Microsoft Authentication Library (MSAL) for Python. https://learn.microsoft.com/en-us/entra/msal/python/

Microsoft Corporation. (2024). OAuth 2.0 Client Credentials Grant Flow. https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow

Microsoft Corporation. (2024). Microsoft Purview Information Protection. https://learn.microsoft.com/en-us/purview/information-protection

Microsoft Corporation. (2024). Azure Entra ID App Registration Quickstart. https://learn.microsoft.com/en-us/entra/identity-platform/quickstart-register-app

Microsoft Corporation. (2024). Azure Entra ID App Registration Quickstart. https://learn.microsoft.com/en-us/entra/identity-platform/quickstart-register-app

McNaughton, C. (2026). Project Brief: Project Sentinel - SECMON1. La Trobe University Capstone Industry Development Program (CSE5IDP).